Code559

Front-End Developer Course

MODULE 13

Capstone

Build a small app from a written requirement, prove every row works, and finish your portfolio.

Free lesson: 559solutions.com/code559/module-13-capstone/

In this packet

1. **The lesson**
2. **Project: Pocket Planner**
3. **Finishing work in Git**
4. **Troubleshooting: six common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn to turn a written requirement into tasks, and to finish each feature with the nine-step completion standard. Then you check the whole site and audit your portfolio. It's the skill real work asks for: building from a short list with no step-by-step guide, and knowing when you are actually done.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. How do you read a requirement table? 5
  - 2. How do you turn a row into tasks? 5
  - 3. What does the finished sample look like? 6
  - 4. What is the nine-step completion standard? 8
  - 5. What is code-origin honesty? 9
  - 6. How do you run a final accessibility pass? 10
  - 7. What is a portfolio audit? 11
  - 8. What comes after the core course? 12
- Exercise 12
- Common misconception 14
- Summary 15
- Check your understanding 15
- Before next module: what comes next 15
- Go further 15
- What to keep 16

Part 2: Project: Pocket Planner 17

Part 3: Finishing work in Git 33

Part 4: Troubleshooting: six common problems 38

Part 5: Self-assessment 42

Part 6: Answers 44

About 559 Solutions 61

© 2026 559 Solutions LLC

PART 1

The lesson

Build a small app from a written requirement, prove every row works, and finish your portfolio.

Time: about 2 hours for this lesson; the full capstone takes longer (estimate)

In this module you will learn:

- How to read a written requirement table and turn each row into tasks
- How to plan before you code: structure first, then look, then behavior
- The Code559 nine-step completion standard: predict, run, explain, modify, debug, retest, commit, push, verify live
- Code-origin honesty: how to mark and explain any line a tool wrote
- How to run a final accessibility pass on the whole site: keyboard, labels, alt text and color contrast
- How to audit your portfolio before you share it
- What to do after the core course, and what this course does not teach

---

Why this matters

Until now every module told you what to build, step by step. Real work does not come like that. It comes as a short written list: "the visitor can do this, the page must do that." Someone hands you the list and expects a working page. The skill this module teaches is turning that list into code without a guide holding your hand.

The second skill is knowing when you are done. "It seems to work" is not done. In this course a feature is done when it has gone through nine steps, ending with you checking it on the live site. That standard is what makes your portfolio something you can stand behind in an interview: you can explain every line, and you can show that every requirement was tested.

This is also the end of the core course. The last part of the module is a whole-site check and a short list of what to do next.

Before you start

1. Finish Modules 11 and 12. You need the Action / Expected / Actual test table from Module 11 and the full Git routine from Module 12.
2. Make sure your personal site is live on GitHub Pages and that `git status` in `my-site` says the working tree is clean.
3. In your course folder (for example `Documents\code559`), create a throwaway folder called `practice-13` and open it in VS Code with **File → Open Folder...**.
4. Have your Module 11 test-table template open in a text file.

© 2026 559 Solutions LLC

> **Checkpoint.** Your live site loads, `my-site` has nothing uncommitted, and you have an empty `practice-13` folder open in VS Code.

Core concepts

1. How do you read a requirement table?

A **requirement** is one thing the finished page must do, written in plain words. A requirement table lists them one per row, with a second column that says how you will know the row is met.

Read the whole table before you write any code. Then read it again, one row at a time, and underline the nouns (the things that must exist on the page) and the verbs (what the visitor does, and what the page does back). The "done when" column is your test. If a row has no "done when," write one before you start; otherwise you cannot tell when you have finished it. Here is a small sample from a task planner called Pocket Planner:

#	Requirement	Done when
1	A visitor can type a task and add it with the Add button or the Enter key. The text box has a visible label.	The task appears at the end of the list and the box empties.
2	An empty or spaces-only task is not added. A message says why.	The list does not change, and the message appears under the form.
3	Each task has a checkbox. Checking it marks the task done, and a line shows how many are done.	The task text is struck through, and the line reads, for example, "1 of 3 done." Unchecking undoes both.
4	One column on a phone. From 700 px wide, the form sits beside the list.	At 390 px and 699 px the panels stack; at 700 px and 1200 px they sit side by side.

Row 2's nouns are "task" and "message." Its verbs are "is not added" and "says why." Its test is "the list does not change, and the message appears."

Underline the nouns and verbs in row 3. How many things must exist on the page for row 3 alone?

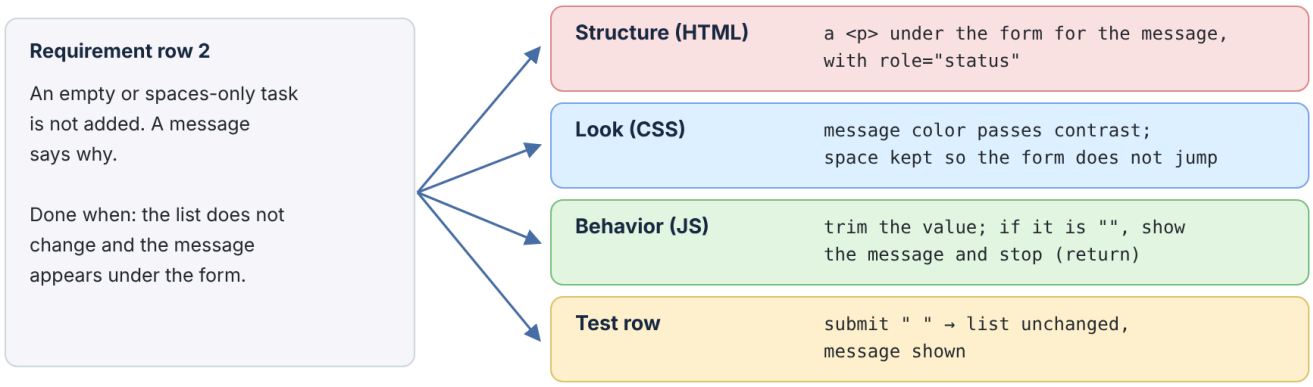
2. How do you turn a row into tasks?

A row is too big to code in one go. You break it into small tasks, one per layer.

For each row ask three questions, in this order. **Structure:** what HTML must exist? **Look:** what CSS does it need? **Behavior:** what JavaScript makes it act? Then add a fourth line: **the test row**, copied straight from "done when" into your Action / Expected / Actual table. Structure comes first because

© 2026 559 Solutions LLC

CSS and JavaScript both need elements to select. Behavior comes last because it is the hardest to test, and it is easier to test on a page that already looks right.



Row 2 as four small tasks. Each one is small enough to finish and check in a few minutes.

Write the four task lines for row 1 of the sample table. Keep each to one sentence.

3. What does the finished sample look like?

To check that the four rows above can be built with what you already know, a minimal Pocket Planner was built for this lesson from those rows. It adds only two small things no row asks for: a header with the planner's name and a short tagline, and the line "No tasks yet." when the list is empty.

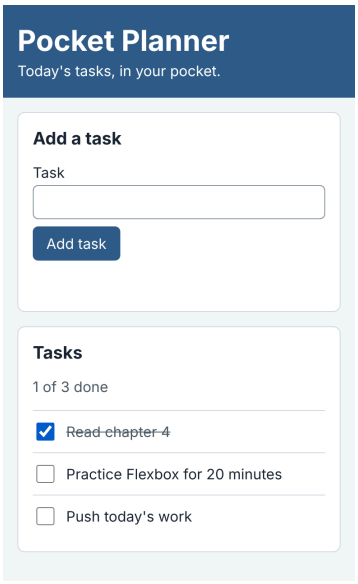
Most of it uses things from earlier modules: a form with a label, `querySelector`, a `submit` listener with `preventDefault()`, `.trim()` and an empty check, `createElement` and `textContent` (no `innerHTML`), `classList`, and one media query. Then each row's "done when" was run as a test. Row 3 needs one thing no earlier module taught: the checkbox's `change` event.

A checkbox fires a **`change`** event each time it is checked or unchecked, by mouse or keyboard (MDN, *HTMLElement: change event*). Inside the listener, the box's **`.checked`** property is `true` if the box is now checked and `false` if not, so the planner gives each new checkbox its own `change` listener when it makes the task. For a page with `<input id="done-box" type="checkbox">` and a `<label>` tied to it:

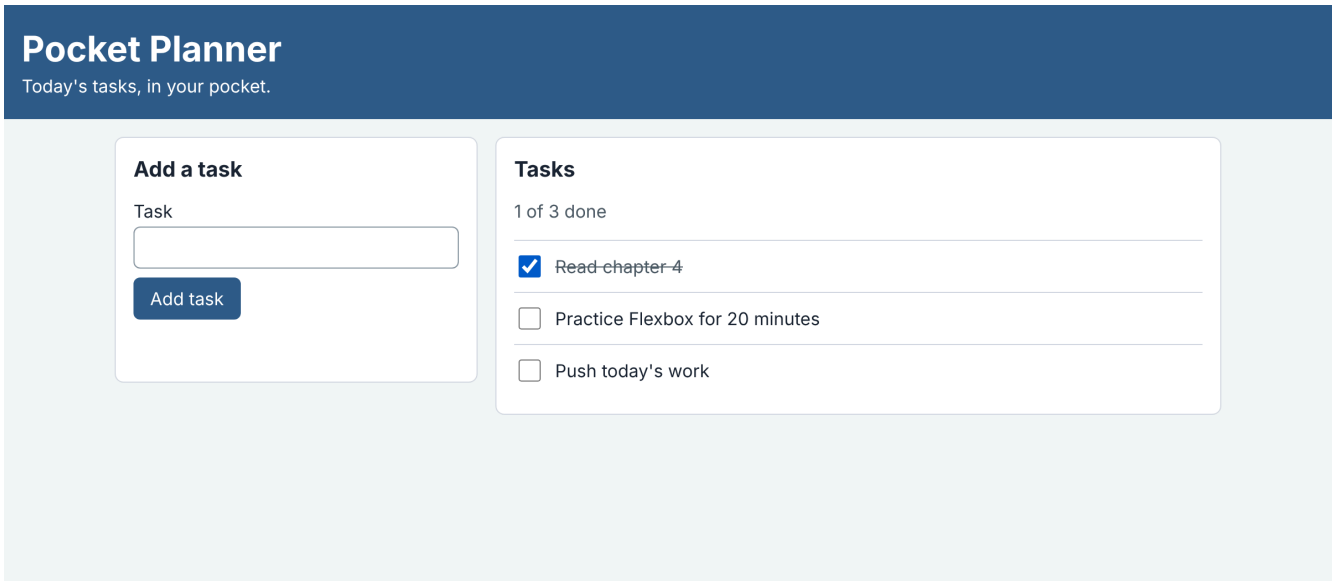
```
const doneBox = document.querySelector("#done-box");
doneBox.addEventListener("change", function () {
  console.log("checked is", doneBox.checked);
});
```


© 2026 559 Solutions LLC

In Chromium, the first click logged `checked is true` and the second logged `checked is false`. Clicking the box's label, or pressing Space on the focused box, did the same.

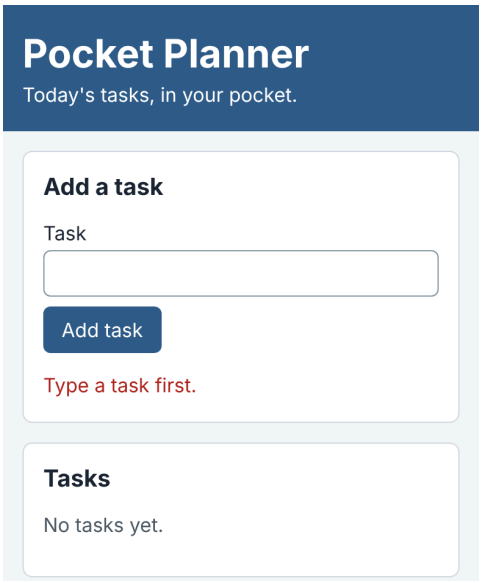


Row 4 at 390 px: one column. Rows 1 and 3 together: three tasks added with the Enter key, the first one checked, and the line reads "1 of 3 done." (real: rendered in Chromium at 390 px, tasks typed and checked by script)



Row 4 at 1200 px: two columns. Measured in Chromium, the grid has one column at 699 px and two at 700 px, which is the boundary the row asks for. (real: rendered in Chromium at 1200 px)

© 2026 559 Solutions LLC



Row 2: three spaces submitted. The list stays empty and the message appears. (real: rendered in Chromium at 390 px)

Look at the 390 px picture and point to the element that proves each of rows 1, 3 and 4.

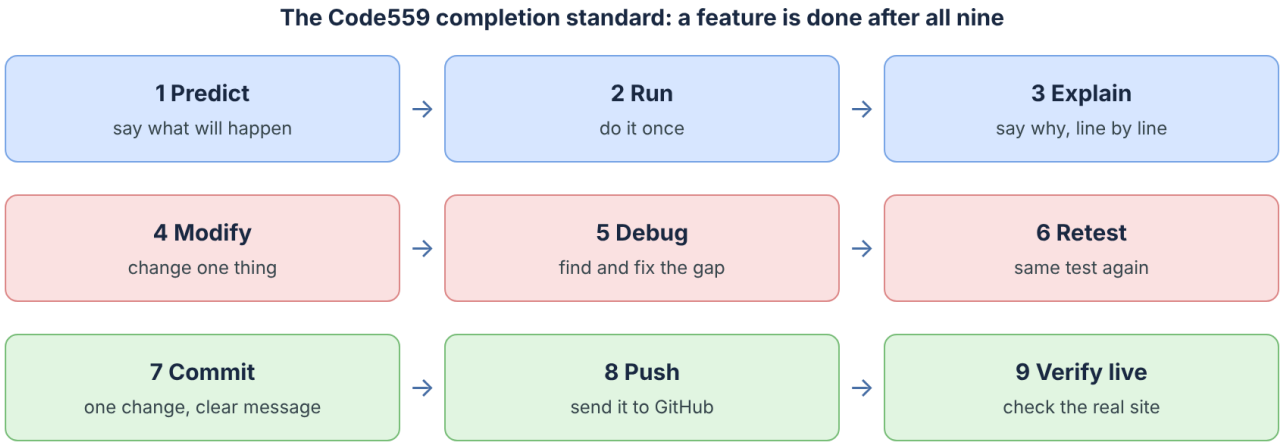
4. What is the nine-step completion standard?

Code559 has one definition of done. A feature is finished when it has passed all nine of these steps, in order.

Predict: before you run new code, write what you expect it to do. **Run:** do it once. **Explain:** say, line by line, why it did what it did. **Modify:** change one thing on purpose and predict again. **Debug:** when actual and expected differ, find the gap with the Module 11 method. **Retest:** run the same test row again. **Commit:** one change, with a message that says what it does. **Push:** send it to GitHub.

© 2026 559 Solutions LLC

Verify live: open the real site, refresh it, and run the test row there. The last step catches the problems that only happen online, such as a file name whose capital letters match on your computer but not on the server.



Read it as three groups: understand it (1–3), make it right (4–6), ship it (7–9). Skipping a group is how "it worked on my computer" happens.

Think of the last feature you built (in a practice folder or on your own site). Which of the nine steps did you skip?

5. What is code-origin honesty?

This course has one honesty rule, set in Module 0: **if a tool wrote a line, you mark it and explain it.** The capstone is where that rule matters most, because it is the piece people will read.

"A tool" means anything that wrote code for you: an AI assistant, a code generator, a snippet you copied from a website. You may use them after this course. What you may not do is present their lines as yours without understanding them. The mark is a comment above the lines, saying where they came from, what they do, and what you changed. If you cannot write the "what they do" part, you are not ready to commit those lines.

```
// Tool-written: the next two lines came from [name the tool or site].
// What they do: count the checked boxes and show "done of total".
// What I changed: renamed the variables to match my page.
```


© 2026 559 Solutions LLC

Search your `my-site` folder for any line you did not type yourself. Mark it with a comment like the one above, or rewrite it.

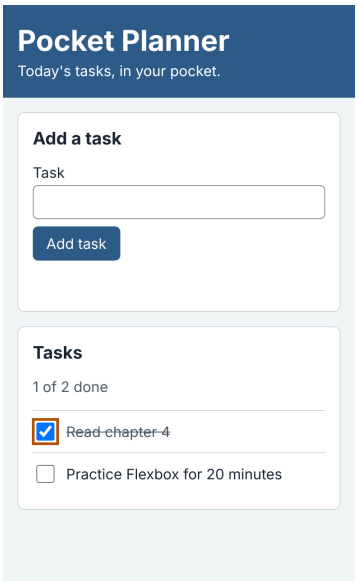
6. How do you run a final accessibility pass?

Before you call the site finished, check it the way a visitor with different needs would use it. Four checks cover most problems.

Keyboard: put the mouse away and Tab through every page (Module 11). Every link, button, text box and checkbox must be reachable and show a visible focus ring, and Space or Enter must work on it. **Labels:** every form field needs a `<label>` tied to it with `for` and `id`; MDN's *HTML: A good basis for accessibility* points out you can then "click the label to select or activate the form element." **Alt text:** every `<img>` needs an `alt`. The same MDN page says the alt "should be brief and concise," and an image that is only decoration gets an empty `alt=""`. **Color contrast:** MDN's *Color contrast* page gives the WCAG AA minimums as **4.5:1** for normal text and **3:1** for large text (at least 18pt, or 14pt bold; W3C's *Understanding SC 1.4.3: Contrast (Minimum)* puts these at about 24px, or about 18.5px bold). Interface parts need contrast too: MDN's *Color contrast* page also asks for at least **3:1** for "user interface components and graphical objects," and that includes a focus ring you style yourself, against the colors next to it (W3C, *Understanding SC 1.4.11: Non-text Contrast*). To check in Chrome or Edge, Chrome's *Make your website more readable* says: open DevTools, click **Inspect** and hover over text; the tooltip shows the contrast ratio, with a warning icon if it is too low. For a whole page at once, the same page names a **Lighthouse** accessibility report.

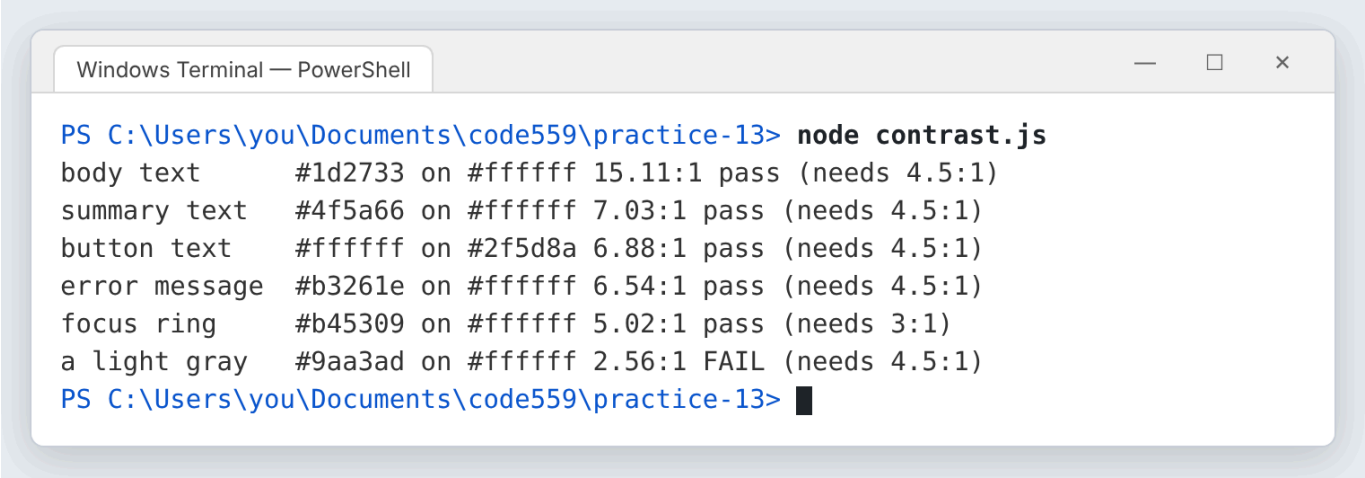
The sample planner passed the keyboard check: from the text box, two presses of Tab reached the first checkbox, and Space checked it.

© 2026 559 Solutions LLC



Keyboard only: Tab, Tab, Space. The ring shows where focus is, and the summary updated, so row 3 works without a mouse. (real: rendered in Chromium at 390 px with real Tab and Space key presses)

The planner's colors were also checked with a short Node script that applies the WCAG contrast formula. You do not need to write such a script; DevTools shows the same kind of ratio.



The four text colors the planner uses pass 4.5:1, and its dark orange focus ring passes the 3:1 it needs. The light gray is there to show what a fail looks like: it is the kind of "soft" gray that looks elegant and is hard to read. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

Open your home page, open DevTools, click Inspect, and hover over your smallest, palest text. Write down the ratio.

7. What is a portfolio audit?

An employer looks at your site for a minute or two, and at your GitHub repositories for less. An audit makes sure that minute goes well.

© 2026 559 Solutions LLC

Open your live site on a phone and on a computer, and go through this list. Every page loads, and every link in the navigation goes where it says. The Console shows no red lines on any page. Every interactive feature has a test plan (Module 11) that you reran this week. Each repository has a README that says what it is, how to see it live, and what you learned. Commit messages say what each change does. Nothing in any repository is a private file, a password, or a half-finished experiment you would not want read.

Treat the audit like the test table: Action ("open Contact on a phone"), Expected ("form fits, label visible"), Actual.

Do the first two checks now, on your phone.

8. What comes after the core course?

You can now build, test, publish and explain a multi-page site with HTML, CSS and JavaScript. That is the core.

A later, optional module (not yet published) will cover a small **Node server** and `fetch`, so a form can send data to a program you wrote and show the reply without leaving the page. After that, many learners want a framework. Frameworks are a separate product, not part of this course, and you should not move to one just because you reached the last module. Move when plain JavaScript feels easy to read and write, and you can explain why a framework would help with a page you have actually built.

A good sign you are ready for more: you can rewrite one of your own functions from a one-sentence description, with the file closed.

Pick one function you wrote in Modules 9 to 11 (or on your own site). Close the file. Rewrite it from memory, then compare.

Exercise

Your hands, about 45 minutes (estimate). Type it, do not paste it. You will turn a three-row requirement into tasks and build **one** row through the first seven of the nine steps, up to commit.

The requirement, for a "Goal counter" page:

#	Requirement	Done when
1	A "+1" button raises a count shown on the page. The count starts at 0.	On load the count reads 0; after three clicks it reads 3.
2	A "Reset" button sets the count back to 0.	After any number of clicks, Reset shows 0.
3	The count stops at 10, and a message says the goal is reached.	The eleventh click leaves the count at 10 and the message appears.

© 2026 559 Solutions LLC

1. **Make tasks.** For each of the three rows, write Structure, Look, Behavior and Test lines (concept 2). That is twelve short lines. Do not code rows 2 or 3; the list is the deliverable for them.

2. **Make the folder.** In `practice-13`, create a folder called `goal-counter` with `index.html` and `script.js`.

3. **Structure for row 1.** Type `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Goal counter</title>
</head>
<body>
  <h1>Goal counter</h1>
  <p>Count: <span id="count">0</span></p>
  <button id="add-button" type="button">+1</button>
  <script src="script.js"></script>
</body>
</html>
```

4. **Behavior for row 1.** Type `script.js`:

```
const countText = document.querySelector("#count");
const addButton = document.querySelector("#add-button");
let count = 0;

function addOne() {
  count = count + 1;
  countText.textContent = count;
}

addButton.addEventListener("click", addOne);
```

5. **Predict, run, explain.** Write your prediction for row 1's test. Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Click +1 three times, and fill in Actual. (In Chromium: 0 on load, 3 after three clicks.) Then explain each line of `script.js` to yourself, out loud, in one sentence.[1]

6. **Modify.** Change `count + 1` to `count + 2`. Predict the count after three clicks. Run it. Change it back.

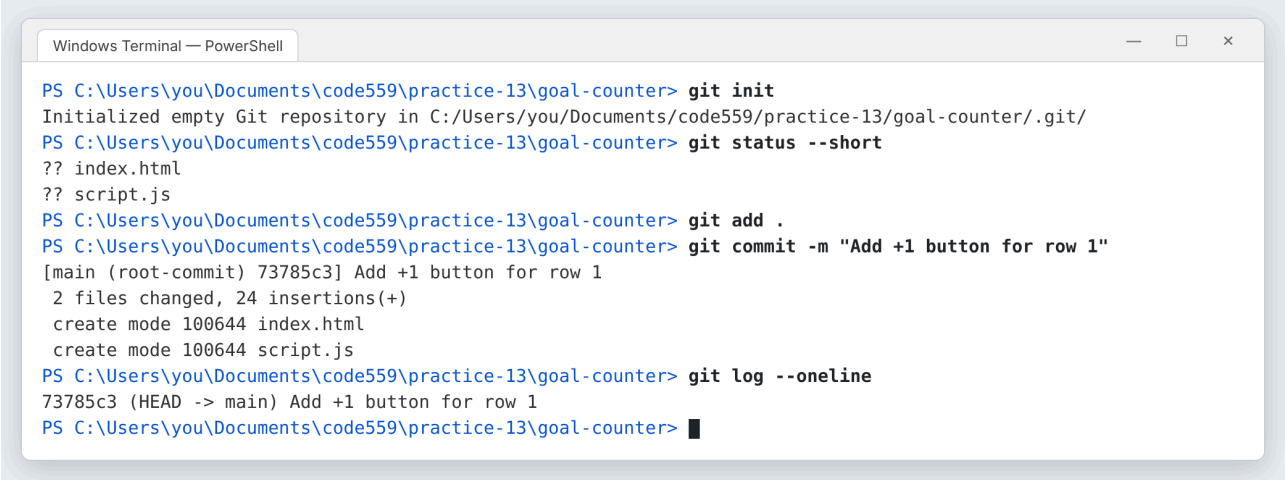
© 2026 559 Solutions LLC

7. **Break it on purpose.** Change the last line to `addButton.addEventListener("click", addOne());` (with the parentheses). Predict first. Then run row 1's test again. In Chromium the count reads **1** as soon as the page loads, clicking does nothing, and there is no red line in the Console. Why? (Hint: Module 7, reference versus call, and Module 9, why `addEventListener()` takes no parentheses.)

8. **Debug and retest.** Remove the parentheses. Run row 1's test again, from a fresh page load. Write the new Actual.

9. **Commit.** Row 1 is one change, so it is one commit. Open the `goal-counter` folder in VS Code with **File → Open Folder...**, then **Terminal → New Terminal**. Check that the prompt ends in `goal-counter>`. Then type these five commands, one at a time:

```
git init
git status --short
git add .
git commit -m "Add +1 button for row 1"
git log --oneline
```



One row, one commit, and the message starts with a verb (Module 12) and names the row. Your commit id will be different. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

This is a throwaway folder, so stop at commit. In the Project, the last two of the nine steps (push and verify live) are required for every row.

Common misconception

> *"If it works on my computer, it's done."*

© 2026 559 Solutions LLC

What would you say? Hint: count how many of the nine steps come after "it works."

Summary

A requirement table lists what the page must do, one row at a time, with a "done when" column that becomes your test. Each row breaks into small tasks in the order structure, look, behavior, plus a test row. The Code559 completion standard says a feature is done only after predict, run, explain, modify, debug, retest, commit, push and verify live. Any line a tool wrote is marked with a comment that says where it came from, what it does and what you changed. A final accessibility pass checks the keyboard, labels, alt text and color contrast (4.5:1 for normal text). A portfolio audit makes sure every page, link, test plan and README is ready for someone else to read. After the core, a later, optional module (not yet published) will cover a Node server with `fetch`; frameworks are a separate product, for when plain JavaScript feels easy.

Check your understanding

1. A requirement row says "The visitor can filter the list." What is missing from it, and what would you add before you start?
2. Why does structure come before behavior when you turn a row into tasks?
3. **What happens if this changes?** In the Exercise, you add parentheses after `addOne` in the listener. What happens on load, what happens on click, and why is there no error?

Before next module: what comes next

- [ ] Your sample tasks: twelve lines for the Goal counter's three rows, and row 1 built, broken, fixed and committed
- [ ] Every page of your live site passed the keyboard, labels, alt text and contrast checks, and you fixed what failed
- [ ] Any tool-written line in `my-site` is marked and explained, or rewritten
- [ ] Your portfolio audit is done on a phone and a computer, with no red lines in any Console
- [ ] Your final changes are saved with the routine: git status → git add . → git commit -m "..." → git push → check the live page
- [ ] You have a short list of links ready to share: your live site, your site's repository, and, if you did the packet projects, your best project's repository
- [ ] You know your next step: more practice from Go further, or a later, optional module on a Node server and `fetch` (not yet published)

Go further

- MDN: *Color contrast* (Accessibility)

© 2026 559 Solutions LLC

- MDN: *HTML: A good basis for accessibility*
- Chrome for Developers: *Make your website more readable* (the contrast checks in DevTools)
- Chrome for Developers: *Pause your code with breakpoints* (for debugging a larger build)
- WebAIM: *Color Contrast Checker* (named on the MDN *Color contrast* page)
- freeCodeCamp: its Responsive Web Design and JavaScript certification projects, for more practice building from a written requirement

---

What to keep

Keep your requirement-to-tasks lines, your test tables, and the three links you would send to an employer. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "Color contrast"; MDN "HTML: A good basis for accessibility"; Chrome for Developers "Make your website more readable"; Chrome for Developers "Pause your code with breakpoints". Checked 2026-10-11: MDN "Color contrast" (3:1 for interface parts); W3C "Understanding SC 1.4.11: Non-text Contrast"; MDN "HTMLElement: change event"; MDN "HTMLInputElement: checked property"; Chrome for Developers "Open Chrome DevTools"; W3C "Understanding SC 1.4.3: Contrast (Minimum)" (1pt = 1.333px). The `change` example, the planner and the Goal counter were run in Chromium 141.0.7390.37 (via Playwright) on Linux. Checked 2026-10-12: Ozubko and MacLeod, "The Production Effect in Memory: Evidence That Distinctiveness Underlies the Benefit," Journal of Experimental Psychology: Learning, Memory, and Cognition 36(6), 2010 (the footnote on saying code out loud).

---

1. **Why out loud?** Most people have recited something to learn it by heart: a phone number, a poem, lines for a play. Words you say aloud stick better than words you read silently; memory researchers call this the production effect (Ozubko and MacLeod, 2010). Explaining code to yourself out loud uses the same idea. It also shows you the line you cannot explain yet, and that line is the one to study next. ↩

© 2026 559 Solutions LLC

PART 2

Project: Pocket Planner

Your hands, about 8 to 10 hours over several sittings (estimate). In the lesson you saw a minimal Pocket Planner built from four rows. Now you build the full one from a **nine-row** requirement, in a new repository of its own. The planner lets a visitor add tasks, tick them off and see how many are done. Above it sits a responsive hero picture. Below it, a Features section is built by JavaScript from an array of objects.

You get far less help than in earlier packets. Each row gives you its tasks, a skeleton or a few hints, and its tests. The rest you write alone, and Part 6, *Answers*, has the sample's full code for you to compare after you finish.

You use what Modules 1 to 12 taught, plus a few small new pieces, each explained where it first appears:

- **HTML:** the page skeleton with the viewport line, `header`, `main`, `section`, a `form` with a `label`, an `input` and a `button`, lists, and an `img` in a `figure` with `figcaption`, `srcset`, `sizes`, `width`, `height` and `alt`; `aria-describedby` for the message
- **CSS:** custom properties on `:root`, the `box-sizing` rule, Grid with `fr` columns and `gap`, mobile-first `min-width` media queries, `:focus-visible` rings, and the `img` rule that lets a picture shrink
- **JavaScript:** `const` and `let`, functions with `return`, `if`, `===`, `querySelector()` and `querySelectorAll()`, `createElement()`, `append()`, `textContent`, `classList`, `addEventListener()` for `submit` and `change`, `event.preventDefault()`, `.value`, `.trim()`, `.focus()`, an array of objects, `for...of`, `.includes()` and `.join()`
- **Git:** the Module 12 routine with `git status --short`, `git diff --stat` and `git diff --staged`, `gh repo create` and GitHub Pages from Module 1, a `.gitignore`, a README, and a `v1.0` tag

There is no `innerHTML` anywhere, no framework, and nothing is saved between visits: when you reload the page, the task list is empty again. That is expected. Saving would need a tool this course does not teach.

The brief

Read the whole brief twice before you type anything. Then read the requirement table a third time, one row at a time, the way Core concept 1 showed you.

When you finish, your course folder holds a new folder, `pocket-planner`, next to `my-site` (not inside it). It is its own Git repository, on GitHub as `pocket-planner`, and live at `https://your-username.github.io/pocket-planner/`. It holds `index.html`, `styles.css`, `script.js`, an `images` folder with three hero files, `test-plan.md`, `README.md`, `explanation.md` and `.gitignore`.

© 2026 559 Solutions LLC

The requirement. Rows 1 to 4 are the lesson's sample rows, word for word.

#	Requirement	Done when
1	A visitor can type a task and add it with the Add button or the Enter key. The text box has a visible label.	The task appears at the end of the list and the box empties.
2	An empty or spaces-only task is not added. A message says why.	The list does not change, and the message appears under the form.
3	Each task has a checkbox. Checking it marks the task done, and a line shows how many are done.	The task text is struck through, and the line reads, for example, "1 of 3 done." Unchecking undoes both.
4	One column on a phone. From 700 px wide, the form sits beside the list.	At 390 px and 699 px the panels stack; at 700 px and 1200 px they sit side by side.
5	Above the planner, a hero picture with a caption. The picture comes in three widths (400, 800 and 1200 px), and the browser picks one.	In device mode (Responsive, which counts 2 device pixels per CSS pixel, as in Module 3) at 390, 800 and 1200 px, the browser loads the 800, 1200 and 1200 file. The picture never makes the page scroll sideways, keeps its space while it loads, and has alt text.
6	Below the planner, a Features section shows one card per feature. The cards are made by JavaScript from an array of objects called `features`, with no `innerHTML`.	The page shows one card per object, in the array's order, each with its title and sentence. One more object in the array means one more card.
7	Each card ends with a line that lists its tags. A feature whose tags include "new" also shows a New badge.	Every card's last line reads "Tags: " and that object's tags with a comma and a space between them. Only the cards tagged "new" show the badge.
8	The feature cards sit in one column on a phone, two columns from 700 px, and four columns from 1000 px.	At 390, 800 and 1200 px the grid has 1, 2 and 4 columns. Nothing scrolls sideways.
9	The whole page passes the final accessibility pass: keyboard, labels, alt text and color contrast.	Tab reaches the text box, the Add task button and each checkbox in order, each with a dark orange ring, and Enter and Space work. Every text color is at least 4.5:1 and the ring at least 3:1. The Console shows no red lines.

The exact words.

- Page title and `<h1>`: **Pocket Planner**, with a short tagline of your own under it (the sample's is *Today's tasks, in your pocket.*)
- `<h2>` headings: **Add a task**, **Tasks** and **Features**
- The label: **Task**; the button: **Add task**

© 2026 559 Solutions LLC

- The message for an empty task: **Type a task first.**
- The done line: **No tasks yet.** for an empty list, otherwise **1 of 3 done** with your own numbers and no period
- The badge: **New**; each tag line starts with **Tags:** and a space

The hero caption and its alt text are yours, because the photo is yours.

The names. Your code and the tests in this packet use these names, so keep them exactly:

- ids: `task-form`, `task-input`, `task-error`, `task-list`, `summary`, `feature-grid`
- classes: `site-header`, `tagline`, `hero`, `layout`, `panel`, `message`, `summary`, `task-list`, `is-done`, `features`, `feature-grid`, `card`, `badge`, `tags`
- files: `images/hero-400.jpg`, `images/hero-800.jpg`, `images/hero-1200.jpg`
- in `script.js`: the array `features`, and the functions `makeTask`, `addTask`, `updateSummary` and `makeCard`

The features. At least four objects in the `features` array, each with three properties in this order: `title` (text), `text` (one short sentence) and `tags` (an array of lowercase words). Write about your planner: what it does and how. Exactly one feature has the tag `"new"`, and at least one feature has two tags.

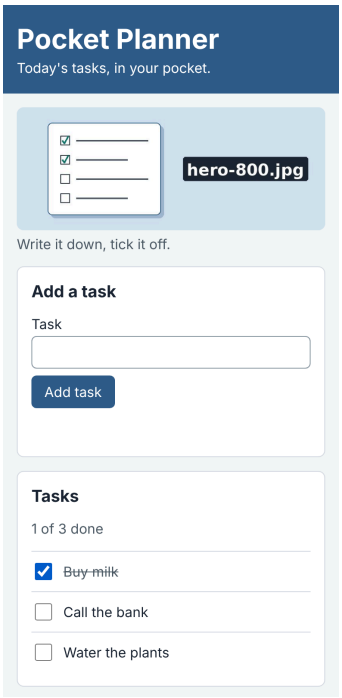
The colors. Nine custom properties on `:root`, used for every color on the page except one: the text box's border is `#8a96a3`, which is 3.01:1 on white, just over the 3:1 that the edge of a form field needs. They are the colors of the lesson's sample planner, plus one for the badge. Each text pair was checked with the WCAG contrast formula, the same one the lesson's `contrast.js` uses:

Name	Value	Used for
`--ink`	`#1d2733`	body text
`--muted`	`#4f5a66`	the done line, the caption, the tag lines, struck-through tasks
`--brand`	`#2f5d8a`	the header and the button
`--paper`	`#ffffff`	panels, cards, text on `--brand` and `--new`
`--page`	`#f4f6f8`	the page background
`--line`	`#d5dbe1`	panel and card borders, lines between tasks
`--focus`	`#b45309`	the focus ring (dark orange)
`--error`	`#b3261e`	the message
`--new`	`#0f766e`	the badge (teal)

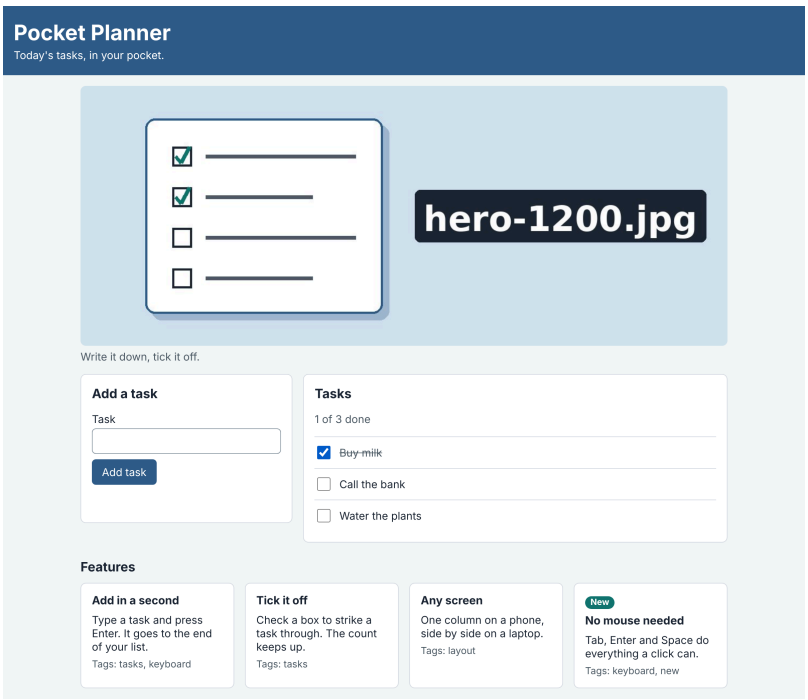
© 2026 559 Solutions LLC

The sample's ratios: `--ink` text 15.11:1 on white; `--muted` text 7.03:1 on white and 6.49:1 on `--page`; white text 6.88:1 on `--brand` and 5.47:1 on `--new`; `--error` text 6.54:1 on white; the `--focus` ring 5.02:1 on white and 4.64:1 on `--page`.

The photo. Pick one photo of your own for the hero: your desk, a notebook, a list on a fridge. Nothing private, and no faces of other people without their permission. A wide shape works best, because the hero spans the page. The sample is a drawing with its file name painted on it, so you can see which file the browser chose; it is 1200 × 480 at the largest size.



The finished planner on a phone, after three tasks were added with the Enter key and the first one was checked. Rendered at 2 pixels per CSS pixel, as in device mode, so the browser chose the 800 file. Only the top of the page is shown. (real: rendered in Chromium at 390 px)



The same page on a laptop: the panels side by side, and four feature cards in a row. Only the last card is tagged "new". (real: rendered in Chromium at 1200 px)

The steps

Type every line. Do not paste. Work on one row at a time, in order, and do not start a row until the one before it is live.

© 2026 559 Solutions LLC

1. **Set up the repository.** Go to Part 3, *Finishing work in Git*, and do steps 1 to 3. They make the `pocket-planner` folder, open it in VS Code, start a repository and commit a `.gitignore`. Then come back here.

2. **Plan on paper.** Turn every row into four task lines, as in Core concept 2: **Structure** (what HTML must exist), **Look** (what CSS it needs), **Behavior** (what JavaScript makes it act) and **Test** (copied from "done when"). Some rows need nothing in one layer: write "none" there. That is 36 short lines. Here is row 1 from the sample's notebook:

   - **Structure:** a form with the label Task, a text box and an Add task button, and an empty list under a Tasks heading.
   - **Look:** the palette, two white panels, a full-width text box, a blue button, a dark orange focus ring.
   - **Behavior:** on submit, stop the reload, read the trimmed text, add it as the last item, empty the box.
   - **Test:** add one task with the button and one with Enter; each is last, and the box empties.

   Write the other 32 lines yourself before you open VS Code. Part 6 has the sample's lines to compare with afterward.

   Then, in VS Code, create `test-plan.md` in `pocket-planner` with a `# Pocket Planner test plan` heading, a one-line note on what the file is, and a `## Row 1` section that holds an empty Action / Expected / Actual table (Module 11). Fill in Action and Expected from your Test line for row 1. Leave Actual empty. Each row adds its own section to this file, and its rows go into the same commit as the code.

3. **The row routine.** Every row below goes through the nine steps from Core concept 4. In this project they look like this:

   1. **Predict:** write the Expected column for the row's tests before you run anything.
   2. **Run:** save, open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window), press **F5**, and do each Action. Keep DevTools open: press **F12** (or **Ctrl+Shift+J**) and click the **Console** tab. Fill in Actual.
   3. **Explain:** explain each new line to yourself, out loud, in one sentence.
   4. **Modify:** make the change the row suggests, predict, run, and undo it.
   5. **Debug:** when Actual and Expected differ, use the Module 11 method: read the red line, check the selector in the Console, and change one thing at a time.
   6. **Retest:** run the row's tests again from a fresh F5, **and the tests of every earlier row**. A new row can break an old one.
   7. **Commit,** 8. **Push** and 9. **Verify live:** Part 3, step 5.

   Steps 7 to 9 are written once, in Part 3, because they are the same for every row. Each row step below ends with **Ship row N**, which means: do Part 3, step 5 for that row.

© 2026 559 Solutions LLC

4. **Row 1: add a task.** *Structure.* Create `index.html` and type:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pocket Planner</title>
  <link rel="stylesheet" href="styles.css">
  <script src="script.js" defer></script>
</head>
<body>
  <header class="site-header">
    <h1>Pocket Planner</h1>
    <p class="tagline">Today's tasks, in your pocket.</p>
  </header>
  <main>
    <div class="layout">
      <section class="panel">
        <h2>Add a task</h2>
        <form id="task-form">
          <label for="task-input">Task</label>
          <input id="task-input" type="text">
          <button type="submit">Add task</button>
        </form>
      </section>
      <section class="panel">
        <h2>Tasks</h2>
        <ul id="task-list" class="task-list"></ul>
      </section>
    </div>
  </main>
</body>
</html>
```

Use your own tagline. The script is in the `<head>` with `defer`, as in Module 9, so it runs after the body exists. The button is a real submit button inside the form, so the click **and** Enter in the text box both fire one `submit` event on the form (Module 10). That one fact covers the "Add button or the Enter key" part of row 1.

Look. Create `styles.css`. Start it with a comment line, the nine custom properties from the brief on `:root`, and the `box-sizing` rule from Module 3. Then type:

```
body {
  margin: 0;
  font-family: system-ui, sans-serif;
```


© 2026 559 Solutions LLC

```
  background: var(--page);
  color: var(--ink);
}

main {
  max-width: 1000px;
  margin: 0 auto;
  padding: 16px;
}

/* Row 1: the panels, the form and the list */
.layout {
  display: grid;
  gap: 16px;
}

button:focus-visible,
input:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 2px;
}

/* Continue alone: .site-header, .site-header h1, .tagline, .panel,
   .panel h2, label, input[type="text"], button, .task-list and
   .task-list li, to match the pictures in the brief. */
```

`system-ui` asks for the computer's own interface font. The ring rule names `input`, not `input[type="text"]`, on purpose: remember that for row 3. For the rest, look at the pictures and use the colors from the table. The text box is as wide as its panel (`width: 100%`), and `font: inherit` on the box and the button makes them use the page's font instead of the browser's smaller default. The list has no bullets and no indent, and each item has a thin `--line` border on top.

Behavior. Create `script.js` and type:

```
// script.js: Pocket Planner

// ===== Find elements =====
const taskForm = document.querySelector("#task-form");
const taskInput = document.querySelector("#task-input");
const taskList = document.querySelector("#task-list");

// ===== Define behavior =====

// makeTask: takes the task text and returns a new list item.
function makeTask(text) {
```


© 2026 559 Solutions LLC

```
  const item = document.createElement("li");
  item.textContent = text;
  return item;
}

// addTask: runs on submit; adds the task to the end of the list.
function addTask(event) {
  event.preventDefault();
  // Continue alone: read the box with .value.trim() into a const text,
  // append makeTask(text) to the list, empty the box, and put the
  // cursor back in it with .focus().
}

// ===== Connect events, and build the page =====
taskForm.addEventListener("submit", addTask);
```

This is the Module 9 pattern: find elements, define behavior, connect events. `makeTask` returns the new item instead of adding it itself, so it can be tested alone in the Console. Write the four missing lines and delete the comment.

Test. Run row 1's tests. The sample added "Buy milk" with the button and "Call the bank" with Enter; each one appeared last, the box emptied, and the cursor was back in the box. Clicking the word **Task** put the cursor in the box, which proves the label is tied to it. Then **think this through:** what will happen if you press Enter with three spaces in the box? Write down your answer, then try it. The sample added an empty item: a thin line with nothing in it. Row 2 exists for exactly this.

Modify. Delete the line that empties the box, predict, run, and put it back.

Ship row 1. Part 3, step 4 is the first push: it creates the repository on GitHub and turns on Pages. Do it now instead of step 5.

5. **Row 2: no empty tasks.** *Structure.* Under the closing `</form>` tag, still inside the first panel, add an empty paragraph for the message: `<p id="task-error" class="message"></p>`. Tie it to the text box by adding `aria-describedby="task-error"` to the `<input>`, as in Module 10. The paragraph is in the HTML from the start, so the attribute always has something to point at.

   Look. Add a `/* Row 2: the message */` comment and a `.message` rule: the `--error` color, `min-height: 1.5em` so the panel does not jump when the message appears, and `margin-bottom: 0`.

   Behavior. Find the message paragraph with a new `const taskError` in the "Find elements" part. In `addTask`, right after the line that reads the text, add the check:

```
  if (text === "") {
    // Continue alone: show "Type a task first." in taskError,
    // put the cursor back in the box, and stop the function.
```


© 2026 559 Solutions LLC

```
  }
  taskError.textContent = "";
```

`return` with nothing after it stops the function at once (Module 7), so nothing below it runs and nothing is added. The line after the `if` block clears an old message once a real task arrives. Update the comment above `addTask`, too: it now also shows a message.

Test. Write row 2's three test rows (empty, three spaces, then a real task) in `test-plan.md`, predict, then run. The sample kept two items after both bad tries, showed **Type a task first.** under the form both times, and cleared the message when "Water the plants" was added. Now rerun row 1's tests: they must still pass.

Modify. Remove `.trim()` from the line that reads the box. Predict what three spaces do now. The sample added an item that held three spaces and showed no message, because `" "` is not `""`. Put `.trim()` back.

Ship row 2.

6. **Row 3: tick it off.** *Structure.* Under the **Tasks** heading, add the done line: `<p id="summary" class="summary"></p>`. Each task now needs more than text. The sample's task item is a `<li>` that holds a `<label>`, and the label holds the checkbox and a `<span>` with the words. A label wrapped around its box needs no `for`: clicking the words checks the box.

   Behavior. Add `const summary` to the "Find elements" part. Then replace `makeTask` with this skeleton, and give it a new comment that says it returns a list item with a checkbox:

```
function makeTask(text) {
  const item = document.createElement("li");
  const label = document.createElement("label");
  const box = document.createElement("input");
  box.type = "checkbox";
  const words = document.createElement("span");
  words.textContent = text;
  box.addEventListener("change", function () {
    // Continue alone: if box.checked, add the class is-done to item;
    // otherwise remove it. Then call updateSummary().
  });
  label.append(box, words);
  item.append(label);
  return item;
}
```

   `box.type = "checkbox"` sets the new input's `type` property, the same as writing `type="checkbox"` in HTML, so the input becomes a checkbox. This is the `change` event from Core concept 3. Each new box gets its own listener, and inside it, `box` and `item` are the box and

© 2026 559 Solutions LLC

the item made on that same call. `append()` can take several elements at once and adds them in order.

Now write `updateSummary()` alone, above `makeTask`, with a comment of its own. It counts the items and the checked boxes, and writes **No tasks yet.** when there are no items, or the "1 of 3 done" line otherwise. Two facts help. `querySelectorAll()` works on any element, not only on `document`, and then it searches only inside that element: `taskList.querySelectorAll("li")` finds the items in the list.

And `:checked` works in a JavaScript selector, as in Module 10, Concept 2, where `document.querySelector('input[name="rating"]:checked')` found the chosen radio: here `"input:checked"` matches only the boxes that are checked now.

Call `updateSummary()` in three places: in the `change` listener, at the end of `addTask`, and once on its own under the `addEventListener` line, so the line is right when the page loads.

Look. Add a `/* Row 3 */` comment and four rules: `.summary` in `--muted`, `.task-list label` as a flex row with a gap (the box beside the words), `.task-list input` at 20 by 20 px, and `.is-done span` with `text-decoration: line-through` (a line through the text) and the `--muted` color. **Think this through:** will the checkboxes get the dark orange ring when you Tab to them? Look at your row 1 ring rule, write down your answer, then Tab and see.

Test. Write row 3's tests from "done when", plus "Load the page" and "Click the words of a task." The sample showed **No tasks yet.** on load and **0 of 3 done** after three tasks. Checking the first box struck it through and showed **1 of 3 done**; unchecking undid both. Clicking the words of the second task checked its box. And yes, the checkboxes had the ring: `input:focus-visible` matches every `input`, checkboxes included. Rerun rows 1 and 2.

Modify. In the `change` listener, delete the `updateSummary();` line. Predict, then check a box. The sample struck the task through but still said **0 of 2 done**. Adding a third task then showed **1 of 3 done**, because `addTask` calls `updateSummary()` too. A count that is right only some of the time is the kind of bug the "Uncheck" test row catches. Put the line back.

Ship row 3.

7. **Row 4: the layout.** This row is one media query, and you write it alone. Put it at the end of `styles.css`, under a `/* Wider screens */` comment, so it comes after the rules it changes (Module 3). Inside, give `.layout` the columns `1fr 2fr` (the list gets twice the room of the form) and `align-items: start`, so the shorter panel does not stretch to match the taller one.

   Test. Turn on device mode (F12, then **Ctrl+Shift+M**) and try 390, 699, 700 and 1200. To be sure of the count, type this in the Console:

   ```
   getComputedStyle(document.querySelector(".layout")).gridTemplateColumns
   ```


© 2026 559 Solutions LLC

One value is one column. The sample measured `358px` at 390, `667px` at 699, `217.328px 434.672px` at 700 and `317.328px 634.672px` at 1200. The boundary is exactly where the row asks, because `min-width` is inclusive. Write the four test rows with the real values in Actual.

Modify. Change `700px` to `800px` and measure again at 700, 799 and 800. The sample stacked at 700 and 799 (`668px`, `767px`) and went side by side at 800. Put back `700px`.

Ship row 4.

8. **Row 5: the hero picture.** *Structure.* Save your photo three times in a new `images` folder inside `pocket-planner`, resized to 400, 800 and 1200 pixels wide, with the names from the brief. Write down the real size of the 1200 file (right-click it, **Properties → Details**), as in the Module 3 packet. Then, as the first thing inside `<main>`, above the layout, write a `<figure class="hero">` with the `<img>` and a `<figcaption>`. You did this in Module 3, so it is yours to write. Use these values:

   - `src`: the 800 file, for a browser that does not read `srcset`
   - `srcset`: all three files with `400w`, `800w` and `1200w`
   - `sizes="(min-width: 1000px) 968px, 100vw"`: the picture is the page width on smaller screens, and at most 968 px wide (1000 px minus the 16 px padding on each side of `main`)
   - `width` and `height`: the real size of your 1200 file (the sample's is `width="1200" height="480"`)
   - `alt`: what the photo shows, in a short sentence

   No `loading="lazy"` here. The hero is on screen the moment the page opens, so waiting would gain nothing.

   Look. Add a `/* Row 5 */` comment and the `img` rule from Module 3 (`max-width: 100%`, `height: auto`, `display: block`), then a `.hero` rule with a bottom margin, rounded corners for `.hero img`, and the caption in `--muted`. Put these above the `/* Wider screens */` block.

   Test. Ask the browser which file it chose. In the Console, type:

   ```
   document.querySelector(".hero img").currentSrc
   ```

   Test in device mode, as in Module 3: its Responsive setting counts 2 device pixels per CSS pixel, so it picks bigger files than a plain window at 100% display scaling. Compare your answers with the 2-pixels column, which is what row 5's "done when" names. A plain window gives a different answer when Windows display scaling is above 100% (for example 150% in **Settings → System → Display**), because the browser then counts more device pixels per CSS pixel. That is not a bug. Press **F5** after every width change: a browser that already has a bigger file may keep showing it. On the sample page, Chromium chose:

© 2026 559 Solutions LLC

Viewport	1 pixel per CSS px	2 pixels per CSS px (device mode)
390 px	hero-400.jpg	hero-800.jpg
800 px	hero-800.jpg	hero-1200.jpg
1200 px	hero-1200.jpg	hero-1200.jpg

At 390 px the picture was 358 px wide and the page did not scroll sideways.

To check that its space is kept while it loads, the sample page was served with each picture held back for 2 seconds: the picture's box was already 143 px tall before the file arrived, and nothing below it moved when it did. You do not need to repeat that check. What you can check is the cause: in the Console, `getComputedStyle(document.querySelector(".hero img")).aspectRatio` gave `auto 1200 / 480`, the shape taken from `width` and `height`. Write row 5's test rows and fill in your own Actual.

Modify. In `sizes`, change `968px` to `400px`. Predict which file loads at 1200 px. The sample loaded `hero-400.jpg` at 1 pixel per CSS px (and `hero-800.jpg` in device mode), and the picture looked soft: `sizes` is a promise, and the browser believed it. Put back `968px`.

Ship row 5. This is the first row with new picture files. Part 3, step 5 says how they show up in Git.

9. **Row 6: cards from an array.** *Structure.* After the layout `<div>`, still inside `<main>`, add a `<section class="features">` with the `<h2>` **Features** and an empty `<ul id="feature-grid" class="feature-grid"></ul>`.

   Behavior. In `script.js`, add `const featureGrid` to "Find elements". Under it, add a `// ===== The data =====` part with your `features` array. The first object is given; continue alone:

```
const features = [
  {
    title: "Add in a second",
    text: "Type a task and press Enter. It goes to the end of your list.",
    tags: ["tasks", "keyboard"]
  },
  // Continue alone: three or more objects of your own, with the same
  // three properties. Exactly one has the tag "new".
];
```

   Then add `makeCard` after `addTask`, with its comment:

```
// makeCard: takes one feature object and returns a new card.
function makeCard(feature) {
  const card = document.createElement("li");
```


© 2026 559 Solutions LLC

```
  card.classList.add("card");
  // Continue alone: an h3 with feature.title and a p with feature.text,
  // both filled with textContent, appended to card in that order.
  return card;
}
```

Last, at the very end of the file, write a `for...of` loop that appends `makeCard(feature)` to `featureGrid` for each feature (Module 9, Concept 9). The tags wait for row 7.

Look. Add a `/* Rows 6 and 7 */` comment above the `/* Wider screens */` block: a `.features h2` rule to match the panel headings, `.feature-grid` with no bullets, no indent, `display: grid` and `gap: 16px`, and `.card` like `.panel` (white, a border, rounded corners, padding). One column for now: the columns are row 8.

Test. Load the page and count the cards. Then, in the Console, type `features.length` and `document.querySelectorAll("#feature-grid li").length`. They must match. The sample gave `4` and `4`, and the cards were in the array's order.

Debug. Here is a mistake that is easy to make when an id has a plural-sounding name. In the `featureGrid` line, change `#feature-grid` to `#features-grid`. Save and press F5. Write down what you expect, then look. On the sample page:

- no cards appeared, and the Console showed one red line, `Uncaught TypeError: Cannot read properties of null (reading 'append')`, with `script.js:102` on the right
- line 102 was the loop's `featureGrid.append(makeCard(feature));`, not the line with the typo
- the planner still worked: adding "Buy milk" showed **0 of 1 done**

Use the Module 11 method. `null` means a `querySelector()` found nothing, and the error shows up later, where the `null` is used. So test the selector itself: in the Console, `document.querySelector("#features-grid")` gave `null`, and `document.querySelector("#feature-grid")` gave the list. The planner kept working because its lines ran before the loop: the error stopped the script only at line 102. Fix the id, F5, and retest rows 1 to 6.

Modify. Under the array, push one more object onto it with `features.push({ ... })`, with a title, a sentence and one tag. Predict the two Console numbers, then F5. The sample showed 5 cards, the new one last. Delete the `push`.

Ship row 6.

10. **Row 7: tags and the New badge.** Write this row alone, inside `makeCard`. Two hints:

    - The tag line is a `<p>` with the class `tags`, appended last. Its text is `"Tags: "` joined with the tags, using `.join(", ")` (Module 8).

© 2026 559 Solutions LLC

- The badge is a `<p>` with the class `badge` and the text `New`, appended **first**, so it sits above the title. Make it only when `feature.tags.includes("new")` is `true`.

In the CSS, `.card .tags` is `--muted` and a little smaller (`0.9rem`). `.badge` has a `--new` background, `--paper` text, bold `0.8rem` type, `display: inline-block` (so the colored box is only as wide as the word) and rounded ends, as in the 1200 px picture.

Test. Read every card's last line against your array, and count the badges. The sample showed `Tags: tasks, keyboard`, `Tags: tasks`, `Tags: layout` and `Tags: keyboard, new`, and one badge, on "No mouse needed" only.

Break it on purpose. In the tag line, delete the parentheses and what is in them, so it reads `"Tags: " + feature.tags.join`. **Think this through:** will there be a red line in the Console? Write down your answer, then F5. In the sample, every card's last line read `Tags: function join() { [native code] }`, and the Console stayed clean.

Without parentheses, `join` is a reference to the method, not a call (Module 7). Joining it to text turns the function itself into text. No error, a wrong page: only reading the result catches it. Put the parentheses back and retest.

Modify. Move the `"new"` tag to a different feature and predict which card gets the badge. Then put it back.

Ship row 7.

11. **Row 8: the feature grid.** Write this row alone, in CSS only. In the `700px` media query, give `.feature-grid` two equal columns (`1fr 1fr`). Then add a second media query, `@media (min-width: 1000px)`, **after** the first one, with four equal columns.

    Test. Measure as in row 4, with `.feature-grid` in place of `.layout`. The sample measured one column (`358px`) at 390, two (`376px` each) at 800, two (`475.5px`) at 999, four (`230px` each) at 1000 and four at 1200. Check that nothing scrolls sideways at any width: the scroll bar at the bottom of the page should not appear.

    Modify. Swap the two media query blocks, so the 1000 px block comes first. Predict the columns at 1200, then measure. The sample had two columns (`476px 476px`) at 1000 and 1200: both blocks were true, they had the same specificity, and the later `1fr 1fr` won (Module 3). Swap them back.

    Ship row 8.

12. **Row 9: the accessibility pass.** This row is the lesson's Core concept 6, done on the whole page. Put the mouse away first. Its test rows go in `test-plan.md` like the others:

    - **Keyboard.** F5, then Tab. Focus goes to the text box first, then the Add task button, then each checkbox. Each one shows the dark orange ring. Enter on the button with an empty box shows the message and puts focus back in the box. Space on a checkbox checks it,

© 2026 559 Solutions LLC

and the done line updates. The sample passed all of these.
- **Labels.** Click the word **Task**: the cursor goes into the box. Click a task's words: its box checks.
- **Alt text.** Right-click the hero, choose **Inspect**, and read its `alt`. It says what the photo shows.
- **Contrast.** In DevTools, click the **Inspect** button (the arrow at the top left of DevTools) and hover over your palest text: a tag line, the caption, the done line. The tooltip shows a contrast ratio. Each must be at least 4.5, with no warning icon. The brief's table lists the sample's ratios.
- **The Console.** After all the tests above, the Console shows no red lines.
- **No `innerHTML`.** Open `script.js`, press **Ctrl+F** and search for `innerHTML`. No results.

Modify. In `:root`, change `--muted` to `#9aa3ad`, the light gray from the lesson. Hover over a tag line with Inspect again. The lesson's `contrast.js` gives that gray 2.56:1 on white, well under 4.5, so the tooltip should show the warning icon. Put back `#4f5a66`.

Debug what you find. If a check fails, fix it, retest that check, and retest every row. The sample passed every check, so its row 9 commit holds only the new section of `test-plan.md`. Yours may hold fixes too, and that is the point of the row.

Ship row 9.

13. **Code origin.** Read all three files from top to bottom. For each line, ask: did I type this myself, from this packet or from my own head? If a tool, a website or a person wrote any line for you, mark it as Core concept 5 shows, with a comment above it:

```
// Tool-written: the next line came from [name the tool or site].
// What it does: [one sentence in your own words].
// What I changed: [what you changed, or "nothing"].
```

In `styles.css` the comment uses `/* */`, and in `index.html` it uses `<!-- -->`. If you cannot write the "What it does" line, rewrite the code in a way you can explain, then commit the change with a message such as `Mark and explain tool-written lines`. Part 3, step 6 has the routine for a change that is not a row.

14. **Write the final explanation.** This is the required last piece of the capstone. In VS Code, create `explanation.md` in `pocket-planner` and fill in this template. Write in full sentences, in your own words, with the files closed for parts 1 and 2. Then open them to check.

```
# How my Pocket Planner works

## 1. What it does
Three or four sentences: what a visitor sees and can do.

## 2. How it works, row by row
```


© 2026 559 Solutions LLC

```
One short paragraph per row, 1 to 9: which file and which lines make
the row work, and why they work.

## 3. The line I was least sure about
Quote the line. Say what you looked up, where (an MDN page title),
and how you checked your new understanding in the browser.

## 4. What breaks if I delete one line
Pick one line. Write your prediction, delete it, run the tests,
write what happened, and put it back.

## 5. Code origin
Which lines a tool or a website wrote, and what you changed.
Write "None" if you typed every line.

## 6. What I would test next
Two or three Action / Expected rows that are not in test-plan.md yet.
```

Part 6 has the sample student's explanation as a model. Read it **after** you write yours.

15. **Check the project.** Tick each box.

    - [ ] `pocket-planner` is its own folder, next to `my-site`, not inside it
    - [ ] Every row's tests in `test-plan.md` pass, with real values in Actual, and every earlier row was retested after each new row
    - [ ] The exact words, ids, classes and file names match the brief
    - [ ] `features` has at least four objects; exactly one has the tag `"new"`; the cards appear in the array's order
    - [ ] There is no `innerHTML` in `script.js`, and every piece of text goes in with `textContent`
    - [ ] The columns are 1 and 2 for the panels at 699 and 700 px, and 1, 2 and 4 for the cards at 390, 800 and 1200 px
    - [ ] Tab, Enter and Space do everything, with a dark orange ring on every control
    - [ ] Every text color passes 4.5:1 in DevTools, and the hero has real alt text
    - [ ] Any tool-written line is marked and explained, and `explanation.md` has all six parts
    - [ ] The extra lines from the Modify, Debug and Break-it steps are gone

16. **Finish in Git.** Go to Part 3 and do steps 7 to 11: the README, the explanation, a full test run on the live site, the `v1.0` tag, and a link from your own site.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

In Modules 1 to 11 you saved each project once, at the end. Here every row is saved on its own: committed, pushed and checked on the live site before the next row starts. That is steps 7 to 9 of the nine, and they are what turn "it works on my computer" into "it works." You use the Module 12 commands, in a new repository of its own, so the planner gets its own page on GitHub and its own live address.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 and 12.

A note on the pictures and numbers. They come from a real run of Git 2.43.0 on Linux, with the sample's files, one commit per row. The pushes went to a local repository standing in for GitHub, so `gh repo create`, the Pages settings and the live checks could not be run for this packet; their steps follow Module 1 and GitHub Docs. Your commit ids, byte sizes and line counts will differ.

1. **Make the folder (NEW: a second project, outside `my-site`).** In File Explorer, open your course folder, `C:\Users\you\Documents\code559`, and create a folder called `pocket-planner`, next to `my-site`. Never put one repository inside another. In VS Code, use **File → Open Folder...** to open `pocket-planner`, then **Terminal → New Terminal**. Check that the prompt ends in `pocket-planner>`.

2. **Start the repository.** Type:

   ```
   git init
   ```

   Git prints `Initialized empty Git repository in` and the path of your folder, ending in `pocket-planner/.git/`. Then type `git status`. It should say `On branch main` and `No commits yet`. If it says `master`, you skipped the `init.defaultBranch` setting from Module 12's Before you start: type `git branch -m main` now.

3. **Commit a `.gitignore` first.** Create `.gitignore` with the three lines from Module 12 (`.vscode/`, `Thumbs.db`, `*.log`), each followed by Enter. Then:

   ```
   git status --short
   git add .
   git commit -m "Add .gitignore"
   ```

   The status line is `?? .gitignore`. The commit's first line starts with `[main (root-commit)`, because it is the first commit in this repository, and the second says `1 file changed, 3 insertions(+)`. Now go back to Project step 2.

4. **Ship row 1: the first push (NEW: a second repository on GitHub).** When row 1 passes its tests on

© 2026 559 Solutions LLC

your computer, read your changes and commit:

```
git status --short
git add .
git diff --staged --stat
git commit -m "Add task form for row 1"
```

The status lists four untracked files: `index.html`, `script.js`, `styles.css` and `test-plan.md`. On the sample, the commit printed `4 files changed, 174 insertions(+)` and one `create mode 100644` line per file. Now put the repository on GitHub, as you did for `my-site` in Module 1:

```
gh repo create pocket-planner --public --source=. --remote=origin --push
```

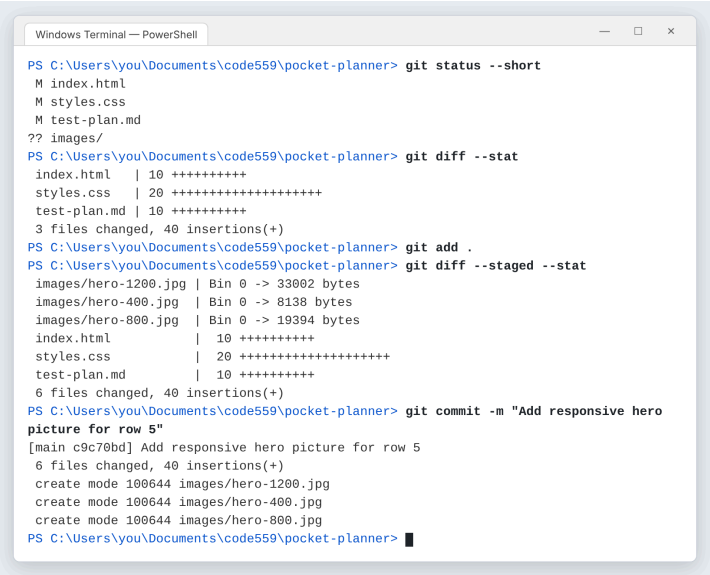
Then turn on Pages for the new repository: open `https://github.com/your-username/pocket-planner`, click **Settings**, then **Pages**. Under **Build and deployment**, choose **Deploy from a branch**, then **main** and **/ (root)**, and click **Save**. GitHub Docs says "It can take up to 10 minutes for changes to your site to publish" (*Creating a GitHub Pages site*). Wait, then open `https://your-username.github.io/pocket-planner/`, press F5, and run row 1's tests there. Your `my-site` address does not change: each repository has its own.

5. **Ship rows 2 to 9: the row routine (NEW: one row, one commit, one live check).** When a row passes its tests on your computer, and every earlier row still passes, do this:

    1. **See what changed.** `git status --short`. You should see `M` beside each file the row changed, and `test-plan.md` among them. A file you did not expect means a change you did not plan: read it before you go on.
    2. **Read the lines.** `git diff --stat`, then `git diff`. Read every `+` and `-` line (press Space for more and **q** to quit).
    3. **Stage, and read again.** `git add .`, then `git diff --staged --stat`. This is the list of what the commit will hold. Check that it is this row and only this row.
    4. **Commit** with the message from the table below, or your own message in the same style: a verb first, then the row.
    5. **Push.** `git push`. The last line ends in `main -> main`.
    6. **Verify live.** Wait a few minutes, open `https://your-username.github.io/pocket-planner/`, and press F5. If you still see the old version, wait and press F5 again. Then run the row's tests on the live page, with DevTools open. Write "live: pass" and the date beside the row in your notebook. Only then start the next row.

    Row 5 is the first row with new files that are not text. **NEW:** a new file does not show in `git diff --stat` until it is staged. `git status --short` lists the pictures as `?? images/`, but `git diff --stat` shows only the three files Git already tracks. After `git add .`, `git diff --staged --stat` lists all six. For a picture, Git shows its size, such as `Bin 0 -> 8138 bytes`, not lines: a picture has no lines to compare.

© 2026 559 Solutions LLC



Row 5 in the sample. Compare the two `--stat` lists: the pictures appear only after `git add .`. If they are missing from the staged list, they will be missing from the live site. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

The sample's commits printed these summaries:

Row	Message	The commit printed
2	`Block empty tasks for row 2`	4 files changed, 25 insertions(+), 2 deletions(-)
3	`Add checkboxes and done count for row 3`	4 files changed, 64 insertions(+), 2 deletions(-)
4	`Place form beside list from 700 px for row 4`	2 files changed, 17 insertions(+)
5	`Add responsive hero picture for row 5`	6 files changed, 40 insertions(+)
6	`Build feature cards from an array for row 6`	4 files changed, 83 insertions(+)
7	`Add tags line and New badge for row 7`	3 files changed, 33 insertions(+), 1 deletion(-)
8	`Lay out features in 1, 2 and 4 columns for row 8`	2 files changed, 20 insertions(+)
9	`Add accessibility checks for row 9`	1 file changed, 14 insertions(+)

Rows 4 and 8 change only `styles.css` and `test-plan.md`. Row 9 changed only `test-plan.md` in the sample, because the sample's pass found nothing to fix.

© 2026 559 Solutions LLC

6. **When a live test fails, or a change is not a row (NEW: fix forward).** Do not start the next row. Find the cause on your computer, fix it, and retest. Then run the routine from step 5 again, with a message that names the fix, such as `Fix hero file name for row 5`. Never change a commit you have already pushed: a new commit on top is the fix, and the history shows both. The most common live-only failure is a file name whose capital letters differ from the code: see Problem 4 in Part 4, *Troubleshooting*. Use the same routine for a change that belongs to no row, such as the code-origin marks from Project step 13.

7. **Add a README.** Create `README.md` with the four headings from Module 12: **What it does**, **Built with**, **What I learned** and **Live site**. Under What it does, also say that the test plan is in `test-plan.md` and the explanation in `explanation.md`. Under Live site, put `https://your-username.github.io/pocket-planner/`. Run the routine with the message `Add README`. On GitHub, the README appears under the file list of the repository.

8. **Commit the final explanation.** When `explanation.md` from Project step 14 is finished, run the routine with the message `Add final explanation`. Open the file on GitHub, which shows its headings as headings, and read it once more there. This is the piece people will read.

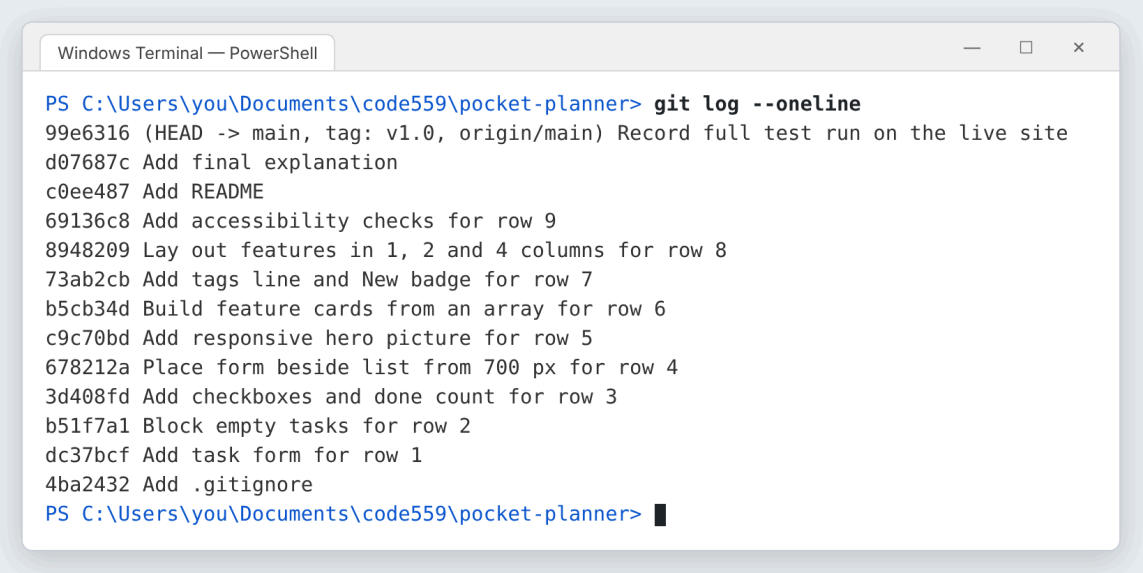
9. **Run the whole test plan on the live site (NEW: a full live run).** One row at a time, from row 1 to row 9, run every test on the live page. Use a phone if you have one, and device mode otherwise. If a row fails, fix it as in step 6 and start the full run again. When every row passes, add one line under the note at the top of `test-plan.md`, such as `Last full run on the live site: (your date), all nine rows pass.`, and run the routine with the message `Record full test run on the live site`. This is the line the lesson's portfolio audit asks for: a test plan you reran this week.

10. **Tag `v1.0` and publish a release.** Check `git log --oneline` first: your newest commit must be the full-run record, at the top. Then tag it and push the tag, as in Module 12:

    ```
    git tag v1.0
    git push origin v1.0
    ```

    The push ends with `* [new tag] v1.0 -> v1.0`. On GitHub, draft a release from the tag: **Choose a tag** `v1.0`, a **Release title** such as `Pocket Planner v1.0`, two sentences of notes, then **Publish release**.

© 2026 559 Solutions LLC



The sample's history: one commit per row, each message naming its row, then the README, the explanation and the full live run. `origin/main` on the top line means GitHub has every commit. (real output, rendered; captured on Linux, with the folder path changed to the course folder)

11. **Link the planner from your own site.** Open `my-site` in VS Code with **File → Open Folder…** and check that the prompt ends in `my-site>`. On your Home page, add one sentence about the planner with a link to `https://your-username.github.io/pocket-planner/`, where it fits your page. Save, open the page, click the link, and press Back. Then save it with the routine: `git status` → `git add .` → `git commit -m "Link to Pocket Planner"` → `git push` → check the live page and click the link there.

> **Checkpoint.** In `pocket-planner`, `git status` says `nothing to commit, working tree clean`, and `git log --oneline` shows a `.gitignore` commit, one commit per row, the README, the explanation and the full live run, with `tag: v1.0` on top. The live planner passes every row of `test-plan.md`. The repository has a README, a v1.0 release and your explanation. Your live site links to the live planner. Those are the links to share: your live site, its repository, and `pocket-planner`.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: six common problems

Each problem has the symptom you see, why it happens, and the fix. Each one was made on purpose in the sample planner and run in Chromium. The line numbers are the sample's, at the row named; yours will differ. After each fix, press F5 and rerun the tests of the row you are on **and of every earlier row**.

1. The page reloads, and the new task is gone

Symptom. You add a task, it flashes, and the list is empty again. The address now ends in `index.html?`, with a question mark.

Why. A form's normal job is to send its data and load a page. The question mark is the form sending itself: its one field has no `name`, so there is nothing after the `?`. Your `submit` listener must call `event.preventDefault()` to stop that (Module 10). Three mistakes in the sample all let the reload happen:

- `event.preventDefault();` missing from `addTask`. No red line.
- The event name misspelled, `"sumbit"`. No red line: the browser accepts any event name and simply waits for an event that never comes.
- `addTask()` with parentheses in the `addEventListener` line. On load, the Console showed `Uncaught TypeError: Cannot read properties of undefined (reading 'preventDefault')` at the `event.preventDefault();` line (line 19 in the row 1 file). `addTask` ran once at load with no event, and no listener was connected.

Fix. Check the listener line first: `taskForm.addEventListener("submit", addTask);`, with the event name spelled right and no parentheses after `addTask`. Then check that `event.preventDefault();` is the first line of `addTask`, and that `addTask` has the parameter `event`.

2. TypeError: Cannot read properties of null

Symptom. A red line in the Console, and part of the page does not work. In the sample:

Message	Where	Cause
`Uncaught TypeError: Cannot read properties of null (reading 'append')`	`script.js:102` (row 6)	`#features-grid` in place of `#feature-grid`: no cards, the planner still works
`Uncaught TypeError: Cannot read properties of null (reading 'addEventListener')`	`script.js:61` (row 3)	`defer` missing from the `<script>` tag in the `<head>`: nothing works, and Add task reloads the page

© 2026 559 Solutions LLC

Why. `querySelector()` returns `null` when it finds nothing (Module 11). The error appears where the `null` is **used**, not where it was made. Without `defer`, the script runs before the body exists, so every `querySelector()` finds nothing (Module 9).

Fix. Look at the word in brackets, such as `'append'`, and at the name just before it on the named line, here `featureGrid`. Find where that name got its value, and test that selector in the Console: `document.querySelector("#feature-grid")`. If it gives `null`, compare the selector with the HTML letter by letter, including the `#`. If every selector gives `null`, check for `defer`.

3. The done count is wrong

Symptom. Checking a box strikes the task through, but the line still says **0 of 2 done**. Or the line counts more tasks than you have, such as **0 of 6 done** after two tasks.

Why. In the first case, the `change` listener does not call `updateSummary()`. The line is only updated when `addTask` runs, so it catches up one task late: in the sample, adding a third task then showed **1 of 3 done**. In the second case, `updateSummary` counts every `<li>` on the page with `document.querySelectorAll("li")`, and from row 6 the feature cards are `<li>` items too. That bug passes every row 3 test and appears only after row 6.

Fix. Call `updateSummary()` at the end of the `change` listener, at the end of `addTask` and once on load. Count with `taskList.querySelectorAll(...)`, which searches only inside the task list. Run the "Uncheck it" test row too: it catches a count that is right only some of the time.

4. The hero picture is missing at some widths, or only online

Symptom. A broken-picture icon (or no picture at all) at one width, a red line in the Console, or a picture that shows on your computer and not on the live site.

Why. Three causes:

- **A wrong name in one `srcset` entry.** The browser loads only the file it chose, so the mistake shows only at the widths that choose that entry. With `images/Hero-400.jpg` in the `srcset` (a capital H), the sample at 390 px showed `Failed to load resource: net::ERR_FILE_NOT_FOUND` for `Hero-400.jpg`.
- **Capital letters, online only.** Windows does not care about capital letters in file names, so on your own computer a wrong capital may still load. The server behind GitHub Pages does care, so the same page fails online. This is the problem Core concept 4 says only "verify live" catches. (This packet could not test on Windows or on GitHub Pages; the run above was on Linux, which, like the server, cares about capital letters.)
- **Pictures not committed.** New files show in `git status --short` as `?? images/` until you `git add` them. If they were never committed, they were never pushed.

© 2026 559 Solutions LLC

Fix. Copy each file name from File Explorer into the `srcset`, all in lowercase, and check all three widths with `currentSrc` (Project step 8). Before you push row 5, check that `git diff --staged --stat` lists all three files (Part 3, step 5).

5. Some cards are missing, or a card shows strange text

Symptom. Fewer cards than objects, with a red line. Or every card's last line shows code instead of tags. In the sample:

What you see	Cause	Where
Two cards, then `Uncaught TypeError: Cannot read properties of undefined (reading 'includes')`	`tag:` in place of `tags:` in the third object	`script.js:89`, the `if` line in `makeCard` (row 7)
Two cards, then `Uncaught TypeError: feature.tags.join is not a function`	`tags: "layout"`, a string, in place of `tags: ["layout"]`	`script.js:101`, the tag line (row 7)
`Tags: function join() { [native code] }` on every card, no red line	`.join` with no parentheses	Project step 10

Why. The loop builds the cards in order and stops at the first error, so the cards before the bad object are made and the rest are not. An object with no `tags` gives `undefined`, and `undefined` has no `.includes()` (Module 8). A string has its own `.includes()`, so the check passes, but it has no `.join()`: only arrays do.

Fix. Count the cards on the page: the bad object is the first one missing. Check that it has all three properties, spelled exactly, and that `tags` has square brackets, even for one tag. Methods need their parentheses: `join(", ")`.

6. The grid has the wrong number of columns

Symptom. Measured with `getComputedStyle(...).gridTemplateColumns` (Project step 7), the panels or the cards have the wrong count at some width. In the sample:

What was wrong	Measured
The `1000px` block above the `700px` block	2 columns at 1000 and 1200 px, not 4
`@media (min-width: 1000)`, with no `px`	2 columns at 1000 and 1200 px: the browser ignores a media query it cannot read
The `700px` block changed to `800px`	Panels still stacked at 700 and 799 px

© 2026 559 Solutions LLC

Why. Rules inside a media query have the same specificity as rules outside it, so the later rule wins when both apply (Module 3). A media query with a mistake in its condition is never true. And the boundary is wherever the number says.

Fix. Keep the base rules first, then the `700px` block, then the `1000px` block, at the end of the file. Write the unit, `px`. Measure at the widths the requirement names, and one pixel each side of each boundary: 699 and 700, 999 and 1000.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. This is the last module of the core course, so aim for at least **3 (Proficient)** on every line before you call the capstone finished and share it. If a line is a 1 or a 2, reread that Core concept and redo its row of the project.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can read a requirement table, underline its nouns and verbs, and write a "done when" for a row that has none	☐	☐	☐	☐
I can turn a row into Structure, Look, Behavior and Test lines, and build it in that order	☐	☐	☐	☐
I can take a row through all nine steps, and retest every earlier row before I commit	☐	☐	☐	☐
I can write test rows from "done when" and fill in Actual with real values: counts, columns and file names	☐	☐	☐	☐
I can build page items from an array of objects with createElement, textContent and append, and no innerHTML	☐	☐	☐	☐
I can find a null selector from a Console error that names a different line than the mistake	☐	☐	☐	☐
I can commit, push and check one row at a time on the live site, reading git diff --staged --stat first	☐	☐	☐	☐
I can run the final accessibility pass: keyboard, labels, alt text and color contrast	☐	☐	☐	☐
I can mark and explain a tool-written line, and explain my whole project in writing	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 9, the typo was in the `featureGrid` line near the top of `script.js`, but the Console named line 102, and the planner above the cards still worked. Explain both facts.
2. Which row surprised you most, on your computer or on the live site? Write the Expected you had, the Actual you got, what you changed, and which of the nine steps caught it.

© 2026 559 Solutions LLC

3. A friend wrote this `updateSummary` in row 3, and it passed every row 3 test. After row 6, what does the done line read on load, after adding two tasks, and after checking one of them? Why did the row 3 tests not catch it, which test does, and what one change fixes it?

```
function updateSummary() {
  const total = document.querySelectorAll("li").length;
  const done = taskList.querySelectorAll("input:checked").length;
  if (total === 0) {
    summary.textContent = "No tasks yet.";
  } else {
    summary.textContent = done + " of " + total + " done";
  }
}
```


© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **"The visitor can filter the list."** It has no "done when," so there is no test and no way to know when the row is finished. It is also vague: filter by what, with which control, and what shows when nothing matches? Before you start, rewrite it and add the test. For example: *"A Show only open tasks checkbox hides the done tasks. Unchecking it shows them all again."* Done when: *"With 1 of 3 done, checking the box leaves 2 tasks on screen; unchecking it shows all 3."* (Core concept 1)

2. **Structure before behavior.** CSS and JavaScript both need elements to select. If the HTML is not there yet, `querySelector()` gives `null`, and the first line that uses it stops the script with a `TypeError` (Module 11). Behavior also comes last because it is the hardest layer to test, and a test is easier to read on a page that already looks right. (Core concept 2)

3. **`addOne()` with parentheses.** On load, the count reads **1**. The parentheses call `addOne` at once, while the listener line runs, so `count` becomes 1 and the page shows it.

   What `addEventListener` receives is not the function but its return value, and `addOne` returns nothing: `undefined`. The browser quietly adds no listener for that, so clicking does nothing. There is no error, because nothing in the code is invalid: a function was called, and a value was passed. The code just does something different from what you meant (Module 7: reference versus call; Module 9: why the listener takes `addOne`, not `addOne()`). Checked in Chromium: 1 on load, still 1 after three clicks, no red line. (Exercise step 7)

Common misconception

"If it works on my computer, it's done." "It works" is step 2 of nine, Run. Seven steps come after it: explain, modify, debug, retest, commit, push and verify live.

Each one catches something "it works" misses. Explaining finds the line you cannot defend. Retesting finds the old row the new code broke. Committing and pushing are what put the work where anyone else can see it: until you push, the live site does not have it at all. And verifying live catches the problems that only happen online, such as a file name whose capital letters match on Windows but not on the server, or a new picture you forgot to commit. In this project, a row is done only when its tests pass on `https://your-username.github.io/pocket-planner/`. (Core concept 4)

© 2026 559 Solutions LLC

Exercise checks

- **Step 1.** Twelve lines. One possible set:

Row	Structure	Look	Behavior	Test
1	A count in a `<span>` that starts at 0, and a +1 button	None yet, or a larger font for the count	On click, add 1 to the count and show it	On load 0; after three clicks 3
2	A Reset button	Spacing between the two buttons	On click, set the count to 0 and show it	After any clicks, Reset shows 0
3	An empty paragraph for the message	The message in a color that passes 4.5:1	Before adding, stop at 10; at 10, show the goal message	The eleventh click leaves 10, and the message appears

- **Step 5.** On load the count reads 0; after three clicks it reads 3. No red lines in the Console.
- **Step 6.** With `count + 2`, three clicks show 6.
- **Step 7.** 1 on load, clicking does nothing, and no red line. See question 3 above for why.
- **Step 8.** From a fresh load: 0, then 3 after three clicks.
- **Step 9.** `git status --short` shows two lines, `?? index.html` and `?? script.js`. The commit printed `2 files changed, 24 insertions(+)` and two `create mode 100644` lines, one per file: 14 lines in `index.html` and 10 in `script.js`. `git log --oneline` shows one commit.

Self-assessment questions

1. **The wrong-looking line number.** The typo was in the `featureGrid` line near the top, but nothing goes wrong on that line. `document.querySelector("#features-grid")` finds nothing, so it returns `null`, and storing `null` in a `const` is allowed. The error comes later, at the first line that **uses** that `null`: `featureGrid.append(...)` inside the loop, line 102. You cannot call `append` on nothing, so Chromium stops with `TypeError: Cannot read properties of null (reading 'append')`.

   The planner still worked because a script runs from the top down, and everything above line 102 had already run: the elements were found, the `submit` listener was connected and `updateSummary()` had run. The error stopped only what came after it, the cards. So the line in the message is where the `null` was used; the cause is wherever that variable got its value. (Project step 9; Module 11)

© 2026 559 Solutions LLC

2. **The row that surprised you.** There is no single answer. A good answer names the row and quotes the Expected you wrote before the run and the Actual you got, on your computer or on the live site. Then it says what you changed, and which step of the nine caught it. For example: a row that passed locally and failed live because a file name had a capital letter, or a retest of row 3 that failed after row 6.

3. **The friend's `updateSummary`.** On load it shows **No tasks yet.**, which passes. After two tasks it shows **0 of 6 done**, and after one check, **1 of 6 done**. `document.querySelectorAll("li")` searches the whole page, and the four feature cards are `<li>` items too. On load the count was right only because `updateSummary()` runs before the loop builds the cards.

   When this function was written, in row 3, there were no cards: it passed every row 3 test (checked in Chromium: **0 of 2 done**, then **1 of 2 done**). Row 6 broke it. Retesting row 3 after row 6 catches it, which is why step 6 of the nine says to rerun every earlier row. The fix is `taskList.querySelectorAll("li")`, which searches inside the list only. (Project steps 6 and 9)

Project checks

- **Step 2.** The sample's lines for rows 2 to 9. Yours will be worded differently; check that each layer is covered.
  - **Row 2.** *Structure:* An empty message paragraph under the form, tied to the box with `aria-describedby`. *Look:* The message in `--error`, with room kept for it. *Behavior:* If the trimmed text is empty: show the message, focus the box, stop; otherwise clear the message. *Test:* Empty and spaces add nothing and show the message; a real task clears it.
  - **Row 3.** *Structure:* A done line under Tasks; each task a label holding a checkbox and a span. *Look:* Box beside the words; struck-through words in `--muted`. *Behavior:* A `change` listener per box toggles `is-done`; `updateSummary` counts. *Test:* No tasks yet; 0 of 3; check gives 1 of 3 and a line; uncheck undoes both.
  - **Row 4.** *Structure:* None. *Look:* One media query from 700 px: `1fr 2fr`. *Behavior:* None. *Test:* 390 and 699 stack; 700 and 1200 side by side.
  - **Row 5.** *Structure:* A figure with the img (srcset, sizes, width, height, alt) and a caption. *Look:* The `img` rule; caption in `--muted`. *Behavior:* None. *Test:* The right file at 390, 800 and 1200; no sideways scroll; alt text.
  - **Row 6.** *Structure:* A Features section with an empty `ul`. *Look:* Grid with a gap; white cards. *Behavior:* The `features` array; `makeCard`; a `for...of` loop that appends. *Test:* One card per object, in order; `features.length` matches.
  - **Row 7.** *Structure:* None (JavaScript makes it). *Look:* Tag line in `--muted`; teal badge. *Behavior:* Tag line with `.join(", ")`; badge if `.includes("new")`. *Test:* Every tag line matches the array; one badge.
  - **Row 8.** *Structure:* None. *Look:* Two columns from 700 px, four from 1000 px, in that order. *Behavior:* None. *Test:* 1, 2 and 4 columns at 390, 800 and 1200.
  - **Row 9.** *Structure, Look and Behavior:* fix anything the checks find. *Test:* Keyboard, labels, alt text, contrast, no red lines, no `innerHTML`.

© 2026 559 Solutions LLC

- **Step 4, continue alone.** The four lines of `addTask` for row 1:

```
const text = taskInput.value.trim();
taskList.append(makeTask(text));
taskInput.value = "";
taskInput.focus();
```

The sample's CSS for row 1. In the full file at the end of this part, these rules sit above `main` and the row 1 comment; the order of these rules does not change the result.

```
.site-header {
  background: var(--brand);
  color: var(--paper);
  padding: 20px 16px;
}

.site-header h1 {
  margin: 0;
}

.tagline {
  margin: 4px 0 0;
}

.panel {
  background: var(--paper);
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 16px;
}

.panel h2 {
  margin-top: 0;
  font-size: 1.2rem;
}

label {
  display: block;
  margin-bottom: 4px;
}

input[type="text"] {
  font: inherit;
  width: 100%;
  padding: 8px;
  margin-bottom: 8px;
```


© 2026 559 Solutions LLC

```
  border: 1px solid #8a96a3;
  border-radius: 6px;
}

button {
  font: inherit;
  padding: 8px 14px;
  border: 1px solid var(--brand);
  border-radius: 6px;
  background: var(--brand);
  color: var(--paper);
  cursor: pointer;
}

.task-list {
  list-style: none;
  padding: 0;
  margin: 0;
}

.task-list li {
  border-top: 1px solid var(--line);
  padding: 10px 0;
}
```

- **Step 5, continue alone.** The three lines inside the `if` block: `taskError.textContent = "Type a task first.";`, `taskInput.focus();` and `return;`.
- **Step 6, continue alone.** Inside the `change` listener:

```
if (box.checked) {
  item.classList.add("is-done");
} else {
  item.classList.remove("is-done");
}
updateSummary();
```

  `updateSummary` and the row 3 CSS are in the full files below.

- **Step 7.** The media query is at the end of the full `styles.css`, under `/* Wider screens */`. At first it holds only the `.layout` rule; row 8 adds the rest.
- **Step 8.** The `<figure class="hero">` is at the top of `<main>` in the full `index.html`, and its CSS is under `/* Row 5: the hero */`.

© 2026 559 Solutions LLC

- **Step 9, continue alone.** The sample's other three objects are in the full `script.js`. `makeCard` for row 6, before row 7 changed it:

```
// makeCard: takes one feature object and returns a new card.
function makeCard(feature) {
  const card = document.createElement("li");
  card.classList.add("card");
  const heading = document.createElement("h3");
  heading.textContent = feature.title;
  const text = document.createElement("p");
  text.textContent = feature.text;
  card.append(heading, text);
  return card;
}
```

  And the loop, the last three lines of the file:

```
for (const feature of features) {
  featureGrid.append(makeCard(feature));
}
```

- **Step 10, alone.** The finished `makeCard` is in the full `script.js`. The badge is made inside an `if`, so features without the tag never get one. A misspelled property name in one object, such as `tag:` instead of `tags:`, stops the loop at that object: see Problem 5 in Part 4, *Troubleshooting*.
- **Step 11, alone.** The second media query is the last block of the full `styles.css`.
- **Step 12.** The sample passed every check. With `--muted` set to `#9aa3ad`, the formula gives 2.56:1 on white, under the 4.5:1 that normal text needs.
- **Step 14.** The sample student's `explanation.md`, as a model. Yours describes your own page, in your own words. Notice that part 2 names real lines and says **why** they work, part 3 names its MDN pages and a Console check, and part 4 reports a real run.

```
# How my Pocket Planner works

## 1. What it does
Pocket Planner is one page with three parts. At the top is a picture of a
notebook with a caption. In the middle, you type a task and add it with the Add
task button or the Enter key; each task gets a checkbox, and a line counts how
many are done. At the bottom, four feature cards are built from an array in
script.js.

## 2. How it works, row by row
Row 1. In index.html, a form holds the label Task, the text box and a submit
```


© 2026 559 Solutions LLC

button, so a click and the Enter key both fire one submit event. In script.js, addTask is the submit listener. It calls event.preventDefault() so the page does not reload, reads the box with .value.trim(), and appends the item that makeTask returns. Then it empties the box and puts the cursor back in it.

Row 2. Before it adds anything, addTask checks text === "". If the text is empty, it writes "Type a task first." into the empty #task-error paragraph under the form, puts the cursor back in the box, and returns, so nothing is added. Because of .trim(), three spaces become "" too. The box has aria-describedby="task-error", so a screen reader reads the message with the field.

Row 3. makeTask builds a label that holds a checkbox and a span with the words, so clicking the words checks the box. Each box gets its own change listener. It adds or removes the class is-done on its item, and the CSS rule .is-done span draws the line through the words. Then it calls updateSummary, which counts the items and the checked boxes inside the list and writes "No tasks yet." or "1 of 3 done".

Row 4. .layout is a grid with one column. Inside @media (min-width: 700px) it gets the columns 1fr 2fr, so from 700 px the form sits beside the wider list. The media query comes after the base rule, so it wins.

Row 5. The hero img has a srcset with three files and their w widths. sizes tells the browser the picture is 968 px wide from 1000 px, and the full width below that, so the browser can pick the smallest file that is big enough. width and height give the shape, 1200 by 480, so the space is kept while the file loads, and img { max-width: 100%; height: auto; } lets it shrink.

Row 6. features is an array of four objects with title, text and tags. A for...of loop at the end of script.js passes each object to makeCard. makeCard makes an li with an h3 and a p, fills them with textContent, and the loop appends the card to #feature-grid. There is no innerHTML, so the text goes in exactly as I wrote it.

Row 7. makeCard also adds a last line, "Tags: " + feature.tags.join(", "). If feature.tags.includes("new") is true, it first makes a p with the class badge and the text New, so the badge sits above the title.

Row 8. .feature-grid is a grid with one column. The 700 px media query gives it two 1fr columns, and a second media query from 1000 px gives it four. The 1000 px block comes after the 700 px block. If it came first, the two-column rule would win at 1200 px.

Row 9. One rule gives every button and every input a 3 px dark orange outline on :focus-visible, so the text box, the button and every checkbox show the same ring. My palest text is the gray --muted color, at 6.49:1 on the page background, and the ring is 5.02:1 on white.

© 2026 559 Solutions LLC

```
## 3. The line I was least sure about
const done = taskList.querySelectorAll("input:checked").length;

I did not know that querySelectorAll could be called on an element instead of on
document, and I had used :checked only with document. I read the MDN pages
"Element: querySelectorAll() method" and ":checked". Called on taskList, it
searches only inside the list, and input:checked matches only the boxes that are
checked right now. To check, I added four tasks, checked the second one, and
typed taskList.querySelectorAll("input:checked").length in the Console. It gave
1.

## 4. What breaks if I delete one line
I picked event.preventDefault(); in addTask. My prediction: the form sends
itself, the page reloads, and the new task is gone. I deleted the line and added
"Buy milk" with Enter. The address ended in index.html? and the list was empty,
with no red line in the Console. I put the line back and reran rows 1 to 3.

## 5. Code origin
None. I typed every line, from the skeletons in the Module 13 packet or from my
own notes.

## 6. What I would test next
| Action | Expected |
| --- | --- |
| At 390 px, add one task that is a single 80-letter word with no spaces | The
word stays inside the Tasks panel, and nothing scrolls sideways |
| Add 30 tasks, then check the last one | "1 of 30 done", and the page scrolls
down, nothing overlaps |
| Add a task, then reload the page with F5 | The list is empty again: nothing is
saved, as the brief says |
```

The sample ran its own part 6 afterward. The 30 tasks gave **1 of 30 done** with nothing overlapping, and the reload emptied the list, as expected. The 80-letter word did **not** pass. At 390 px the page became 806 px wide and scrolled sideways, and at 1200 px the form panel was squeezed to 162 px while the list took 790 px. Row 8 tests normal text; no row tests a very long word. That is what "what I would test next" is for.

Write it down as a GitHub Issue on the `pocket-planner` repository (Module 12) and fix it next time. The fix that worked on the sample is one rule, `.task-list span { overflow-wrap: anywhere; }` (MDN, *overflow-wrap*): with it, the page stayed 390 px wide and the panels went back to 317 px and 635 px at 1200. The value `break-word` did not help in this grid.

- **Step 15.** The finished sample files. They are the files of the sample's row 9 commit; the README and the explanation came after.

© 2026 559 Solutions LLC

`index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Pocket Planner</title>
  <link rel="stylesheet" href="styles.css">
  <script src="script.js" defer></script>
</head>
<body>
  <header class="site-header">
    <h1>Pocket Planner</h1>
    <p class="tagline">Today's tasks, in your pocket.</p>
  </header>
  <main>
    <figure class="hero">
      <img src="images/hero-800.jpg"
           srcset="images/hero-400.jpg 400w,
                   images/hero-800.jpg 800w,
                   images/hero-1200.jpg 1200w"
           sizes="(min-width: 1000px) 968px, 100vw"
           width="1200" height="480"
           alt="An open notebook with a short list of tasks, two of them
ticked">
      <figcaption>Write it down, tick it off.</figcaption>
    </figure>
    <div class="layout">
      <section class="panel">
        <h2>Add a task</h2>
        <form id="task-form">
          <label for="task-input">Task</label>
          <input id="task-input" type="text" aria-describedby="task-error">
          <button type="submit">Add task</button>
        </form>
        <p id="task-error" class="message"></p>
      </section>
      <section class="panel">
        <h2>Tasks</h2>
        <p id="summary" class="summary"></p>
        <ul id="task-list" class="task-list"></ul>
      </section>
    </div>
    <section class="features">
      <h2>Features</h2>
      <ul id="feature-grid" class="feature-grid"></ul>
```


© 2026 559 Solutions LLC

```
    </section>
  </main>
</body>
</html>
```

`styles.css`:

```
/* styles.css: Pocket Planner */

:root {
  --ink: #1d2733;
  --muted: #4f5a66;
  --brand: #2f5d8a;
  --paper: #ffffff;
  --page: #f4f6f8;
  --line: #d5dbe1;
  --focus: #b45309;
  --error: #b3261e;
  --new: #0f766e;
}

*, *::before, *::after {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: var(--page);
  color: var(--ink);
}

.site-header {
  background: var(--brand);
  color: var(--paper);
  padding: 20px 16px;
}

.site-header h1 {
  margin: 0;
}

.tagline {
  margin: 4px 0 0;
}
```


© 2026 559 Solutions LLC

```
main {
  max-width: 1000px;
  margin: 0 auto;
  padding: 16px;
}

/* Row 1: the panels, the form and the list */
.layout {
  display: grid;
  gap: 16px;
}

.panel {
  background: var(--paper);
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 16px;
}

.panel h2 {
  margin-top: 0;
  font-size: 1.2rem;
}

label {
  display: block;
  margin-bottom: 4px;
}

input[type="text"] {
  font: inherit;
  width: 100%;
  padding: 8px;
  margin-bottom: 8px;
  border: 1px solid #8a96a3;
  border-radius: 6px;
}

button {
  font: inherit;
  padding: 8px 14px;
  border: 1px solid var(--brand);
  border-radius: 6px;
  background: var(--brand);
  color: var(--paper);
  cursor: pointer;
}
```


© 2026 559 Solutions LLC

```
button:focus-visible,
input:focus-visible {
  outline: 3px solid var(--focus);
  outline-offset: 2px;
}

.task-list {
  list-style: none;
  padding: 0;
  margin: 0;
}

.task-list li {
  border-top: 1px solid var(--line);
  padding: 10px 0;
}

/* Row 2: the message */
.message {
  color: var(--error);
  min-height: 1.5em;
  margin-bottom: 0;
}

/* Row 3: the checkboxes and the done count */
.summary {
  color: var(--muted);
  margin-top: 0;
}

.task-list label {
  display: flex;
  gap: 10px;
  align-items: center;
  margin: 0;
}

.task-list input {
  width: 20px;
  height: 20px;
}

.is-done span {
  text-decoration: line-through;
  color: var(--muted);
}
```


© 2026 559 Solutions LLC

```
/* Row 5: the hero */
img {
  max-width: 100%;
  height: auto;
  display: block;
}

.hero {
  margin: 0 0 16px;
}

.hero img {
  border-radius: 8px;
}

.hero figcaption {
  color: var(--muted);
  margin-top: 6px;
}

/* Rows 6 and 7: the feature cards */
.features h2 {
  font-size: 1.2rem;
  margin: 24px 0 12px;
}

.feature-grid {
  list-style: none;
  padding: 0;
  margin: 0;
  display: grid;
  gap: 16px;
}

.card {
  background: var(--paper);
  border: 1px solid var(--line);
  border-radius: 8px;
  padding: 16px;
}

.card h3 {
  margin: 0 0 8px;
  font-size: 1.05rem;
}
```


© 2026 559 Solutions LLC

```
.card p {
  margin: 0 0 8px;
}

.card .tags {
  color: var(--muted);
  font-size: 0.9rem;
  margin: 0;
}

.badge {
  display: inline-block;
  background: var(--new);
  color: var(--paper);
  font-size: 0.8rem;
  font-weight: bold;
  padding: 2px 8px;
  border-radius: 999px;
}

/* Wider screens */
@media (min-width: 700px) {
  .layout {
    grid-template-columns: 1fr 2fr;
    align-items: start;
  }

  .feature-grid {
    grid-template-columns: 1fr 1fr;
  }
}

@media (min-width: 1000px) {
  .feature-grid {
    grid-template-columns: 1fr 1fr 1fr 1fr;
  }
}
```

`script.js`:

```
// script.js: Pocket Planner

// ===== Find elements =====
const taskForm = document.querySelector("#task-form");
const taskInput = document.querySelector("#task-input");
const taskError = document.querySelector("#task-error");
```


© 2026 559 Solutions LLC

```
const taskList = document.querySelector("#task-list");
const summary = document.querySelector("#summary");
const featureGrid = document.querySelector("#feature-grid");

// ===== The data =====
const features = [
  {
    title: "Add in a second",
    text: "Type a task and press Enter. It goes to the end of your list.",
    tags: ["tasks", "keyboard"]
  },
  {
    title: "Tick it off",
    text: "Check a box to strike a task through. The count keeps up.",
    tags: ["tasks"]
  },
  {
    title: "Any screen",
    text: "One column on a phone, side by side on a laptop.",
    tags: ["layout"]
  },
  {
    title: "No mouse needed",
    text: "Tab, Enter and Space do everything a click can.",
    tags: ["keyboard", "new"]
  }
];

// ===== Define behavior =====

// updateSummary: shows how many tasks are done, or "No tasks yet."
function updateSummary() {
  const total = taskList.querySelectorAll("li").length;
  const done = taskList.querySelectorAll("input:checked").length;
  if (total === 0) {
    summary.textContent = "No tasks yet.";
  } else {
    summary.textContent = done + " of " + total + " done";
  }
}

// makeTask: takes the task text and returns a new list item with a checkbox.
function makeTask(text) {
  const item = document.createElement("li");
  const label = document.createElement("label");
  const box = document.createElement("input");
  box.type = "checkbox";
```


© 2026 559 Solutions LLC

```
  const words = document.createElement("span");
  words.textContent = text;
  box.addEventListener("change", function () {
    if (box.checked) {
      item.classList.add("is-done");
    } else {
      item.classList.remove("is-done");
    }
    updateSummary();
  });
  label.append(box, words);
  item.append(label);
  return item;
}

// addTask: runs on submit; adds the task, or shows a message when it is empty.
function addTask(event) {
  event.preventDefault();
  const text = taskInput.value.trim();
  if (text === "") {
    taskError.textContent = "Type a task first.";
    taskInput.focus();
    return;
  }
  taskError.textContent = "";
  taskList.append(makeTask(text));
  taskInput.value = "";
  taskInput.focus();
  updateSummary();
}

// makeCard: takes one feature object and returns a new card.
function makeCard(feature) {
  const card = document.createElement("li");
  card.classList.add("card");
  if (feature.tags.includes("new")) {
    const badge = document.createElement("p");
    badge.classList.add("badge");
    badge.textContent = "New";
    card.append(badge);
  }
  const heading = document.createElement("h3");
  heading.textContent = feature.title;
  const text = document.createElement("p");
  text.textContent = feature.text;
  const tagLine = document.createElement("p");
  tagLine.classList.add("tags");
```


© 2026 559 Solutions LLC

```
  tagLine.textContent = "Tags: " + feature.tags.join(", ");
  card.append(heading, text, tagLine);
  return card;
}

// ===== Connect events, and build the page =====
taskForm.addEventListener("submit", addTask);
updateSummary();

for (const feature of features) {
  featureGrid.append(makeCard(feature));
}
```


© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC